

```
1  园林景观三维可视化设计平台 V1.0
2  [padding]
3  # apps/api/main.py
4  # -*- coding: utf-8 -*-
5  from __future__ import annotations
6  [padding]
7  import mimetypes
8  import os
9  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
10 from pathlib import Path
11 [padding]
12 try:
13     from .domain.auth import authenticate, issue_token
14     from .domain.csv_tools import parse_csv
15     from .domain.exporter import export_all
16     from .domain.errors import ApiError
17     from .domain.reporting import build_markdown_report
18     from .domain.router import Router, build_request, handle_api_error,
19 json_response, text_response
20     from .domain.static_files import build_manifest, build_static_response
21     from .domain.store import ProjectStore
22     from .domain.version_diff import diff_snapshots
23 except ImportError: # run as script: `python apps/api/main.py`
24     import sys
25 [padding]
26 PROJECT_ROOT_FALLBACK = Path(__file__).resolve().parents[2]
27 sys.path.insert(0, str(PROJECT_ROOT_FALLBACK))
28 from apps.api.domain.auth import authenticate, issue_token
29 from apps.api.domain.csv_tools import parse_csv
30 from apps.api.domain.exporter import export_all
31 from apps.api.domain.errors import ApiError
32 from apps.api.domain.reporting import build_markdown_report
33 from apps.api.domain.router import Router, build_request,
34 handle_api_error, json_response, text_response
35     from apps.api.domain.static_files import build_manifest,
36 build_static_response
37     from apps.api.domain.store import ProjectStore
38     from apps.api.domain.version_diff import diff_snapshots
39 [padding]
40 [padding]
41 PROJECT_ROOT = Path(__file__).resolve().parents[2]
42 WEB_ROOT = PROJECT_ROOT / "apps" / "web"
43 STORE = ProjectStore(PROJECT_ROOT)
44 ROUTER = Router()
45 [padding]
46 [padding]
47 def _read_body(handler: BaseHTTPRequestHandler) -> bytes:
48     length = int(handler.headers.get("Content-Length") or "0")
49     if length <= 0:
50         return b""
```

```
51     return handler.rfile.read(length)
52 [padding]
53 [padding]
54 def _safe_join(root: Path, rel: str) -> Path | None:
55     rel = rel.lstrip("/").replace("/", os.sep)
56     target = (root / rel).resolve()
57     try:
58         target.relative_to(root.resolve())
59     except ValueError:
60         return None
61     return target
62 [padding]
63 [padding]
64 class Handler(BaseHTTPRequestHandler):
65     server_version = "Garden3D/1.0"
66 [padding]
67     def log_message(self, fmt: str, *args) -> None:
68         return
69 [padding]
70     def do_GET(self) -> None:
71         if self.path.startswith("/api/"):
72             return self._handle_api()
73 [padding]
74         resp = build_static_response(root=WEB_ROOT, request_path=self.path,
75 fallback="index.html")
76         self._send(resp.status, resp.headers, resp.body)
77 [padding]
78     def do_POST(self) -> None:
79         if self.path.startswith("/api/"):
80             return self._handle_api()
81         self.send_response(405)
82         self.end_headers()
83 [padding]
84     def do_PUT(self) -> None:
85         if self.path.startswith("/api/"):
86             return self._handle_api()
87         self.send_response(405)
88         self.end_headers()
89 [padding]
90     def _send(self, status: int, headers: dict[str, str], body: bytes) ->
91 None:
92         self.send_response(status)
93         for k, v in headers.items():
94             self.send_header(k, v)
95         self.end_headers()
96         if body:
97             self.wfile.write(body)
98 [padding]
99     def _handle_api(self) -> None:
100     try:
```

```
101         req = build_request(self.command, self.path, {k: str(v) for k, v
102 in self.headers.items()}, _read_body(self))
103         handler, params = ROUTER.match(req)
104         status, headers, body = handler(req, params)
105         self._send(status, headers, body)
106     except ApiError as exc:
107         status, headers, body = handle_api_error(exc)
108         self._send(status, headers, body)
109     except Exception as exc:
110         status, headers, body = json_response(500, {"ok": False,
111 "error": "internal_error", "message": str(exc)})
112         self._send(status, headers, body)
113 [padding]
114 [padding]
115 def _require_demo_role(req, allowed: set[str]) -> None:
116     auth = (req.headers.get("Authorization") or "").strip()
117     if not auth.startswith("Bearer demo-"):
118         # Demo mode: allow missing auth for reads.
119         return
120     parts = auth.replace("Bearer ", "", 1).split("-")
121     role = parts[1] if len(parts) >= 2 else ""
122     if role and role not in allowed:
123         raise ApiError(403, "forbidden", "forbidden")
124 [padding]
125 [padding]
126 def _api_health(req, params):
127     return json_response(200, {"ok": True})
128 [padding]
129 [padding]
130 def _api_login(req, params):
131     ctype = (req.headers.get("Content-Type") or "").lower()
132     username = ""
133     password = ""
134     if "application/json" in ctype:
135         payload = req.json()
136         username = str(payload.get("username") or "").strip()
137         password = str(payload.get("password") or "").strip()
138     else:
139         form = req.form()
140         username = (form.get("username") or "").strip()
141         password = (form.get("password") or "").strip()
142     user = authenticate(username, password)
143     token = issue_token(user)
144     return json_response(200, {"ok": True, "token": token, "role": user.role})
145 )
146 [padding]
147 [padding]
148 def _api_sites(req, params):
149     return json_response(200, {"ok": True, "items": STORE.list_sites()})
150 [padding]
```

```
151 [padding]
152 def _api_plans(req, params):
153     return json_response(200, {"ok": True, "items": STORE.list_plans()})
154 [padding]
155 [padding]
156 def _api_versions(req, params):
157     return json_response(200, {"ok": True, "items": STORE.list_versions()})
158 [padding]
159 [padding]
160 def _api_plants(req, params):
161     return json_response(200, {"ok": True, "items": STORE.list_plants()})
162 [padding]
163 def _api_create_plant(req, params):
164     _require_demo_role(req, {"admin", "designer"})
165     item = STORE.create_plant(req.json())
166     return json_response(201, {"ok": True, "item": item})
167 [padding]
168 [padding]
169 def _api_update_plant(req, params):
170     _require_demo_role(req, {"admin", "designer"})
171     plant_id = params["plantId"]
172     item = STORE.update_plant(plant_id, req.json())
173     return json_response(200, {"ok": True, "item": item})
174 [padding]
175 [padding]
176 def _api_delete_plant(req, params):
177     _require_demo_role(req, {"admin"})
178     plant_id = params["plantId"]
179     STORE.delete_plant(plant_id)
180     return json_response(200, {"ok": True})
181 [padding]
182 [padding]
183 def _api_import_plants(req, params):
184     _require_demo_role(req, {"admin", "designer"})
185     payload = req.json()
186     text = str(payload.get("csv") or "")
187     rows = parse_csv(text)
188     result = STORE.import_plants_csv(rows)
189     return json_response(200, {"ok": True, "result": result})
190 [padding]
191 [padding]
192 def _api_materials(req, params):
193     return json_response(200, {"ok": True, "items": STORE.list_materials()})
194 [padding]
195 def _api_create_material(req, params):
196     _require_demo_role(req, {"admin", "designer"})
197     item = STORE.create_material(req.json())
198     return json_response(201, {"ok": True, "item": item})
199 [padding]
200 [padding]
```

```
201 def _api_update_material(req, params):
202     _require_demo_role(req, {"admin", "designer"})
203     item = STORE.update_material(params["materialId"], req.json())
204     return json_response(200, {"ok": True, "item": item})
205 [padding]
206 [padding]
207 def _api_delete_material(req, params):
208     _require_demo_role(req, {"admin"})
209     STORE.delete_material(params["materialId"])
210     return json_response(200, {"ok": True})
211 [padding]
212 [padding]
213 def _api_lights(req, params):
214     return json_response(200, {"ok": True, "items": STORE.list_lights()})
215 [padding]
216 def _api_create_light(req, params):
217     _require_demo_role(req, {"admin", "designer"})
218     item = STORE.create_light(req.json())
219     return json_response(201, {"ok": True, "item": item})
220 [padding]
221 [padding]
222 def _api_update_light(req, params):
223     _require_demo_role(req, {"admin", "designer"})
224     item = STORE.update_light(params["lightId"], req.json())
225     return json_response(200, {"ok": True, "item": item})
226 [padding]
227 [padding]
228 def _api_delete_light(req, params):
229     _require_demo_role(req, {"admin"})
230     STORE.delete_light(params["lightId"])
231     return json_response(200, {"ok": True})
232 [padding]
233 [padding]
234 def _api_issues(req, params):
235     return json_response(200, {"ok": True, "items": STORE.list_issues()})
236 [padding]
237 [padding]
238 def _api_create_issue(req, params):
239     _require_demo_role(req, {"admin", "designer", "reviewer"})
240     version_id = params["versionId"]
241     payload = req.json()
242     issue = STORE.create_issue(version_id, payload)
243     return json_response(201, {"ok": True, "item": issue})
244 [padding]
245 [padding]
246 def _api_update_issue(req, params):
247     _require_demo_role(req, {"admin", "designer"})
248     issue_id = params["issueId"]
249     payload = req.json()
250     issue = STORE.update_issue(issue_id, payload)
```

```
251     return json_response(200, {"ok": True, "item": issue})
252 [padding]
253 [padding]
254 def _api_pavings(req, params):
255     return json_response(200, {"ok": True, "items": STORE.list_pavings()})
256 [padding]
257 [padding]
258 def _api_create_paving(req, params):
259     _require_demo_role(req, {"admin", "designer"})
260     plan_id = params["planId"]
261     payload = req.json()
262     paving = STORE.create_paving(plan_id, payload)
263     return json_response(201, {"ok": True, "item": paving})
264 [padding]
265 [padding]
266 def _api_export(req, params):
267     bundle = export_all(STORE)
268     kind = (req.query.get("kind") or ["all"])[0]
269     if kind == "plants":
270         return text_response(200, bundle.plants_csv, content_type="text/csv;
271 charset=utf-8")
272     if kind == "materials":
273         return text_response(200, bundle.materials_csv, content_type="
274 text/csv; charset=utf-8")
275     if kind == "lights":
276         return text_response(200, bundle.lights_csv, content_type="text/csv;
277 charset=utf-8")
278     if kind == "all":
279         combined = (
280             "# plants\n"
281             + bundle.plants_csv
282             + "\n# materials\n"
283             + bundle.materials_csv
284             + "\n# lights\n"
285             + bundle.lights_csv
286         )
287         return text_response(200, combined, content_type="text/plain;
288 charset=utf-8")
289     raise ApiError(400, "bad_request", "unknown export kind")
290 [padding]
291 def _api_diff(req, params):
292     before_id = (req.query.get("before") or [""])[0].strip()
293     after_id = (req.query.get("after") or [""])[0].strip()
294     if not before_id or not after_id:
295         raise ApiError(400, "bad_request", "before/after are required")
296     before = STORE.get_version(before_id).get("snapshot") or {}
297     after = STORE.get_version(after_id).get("snapshot") or {}
298     if not isinstance(before, dict) or not isinstance(after, dict):
299         raise ApiError(400, "bad_request", "snapshot must be object")
300     return json_response(200, {"ok": True, "diff": diff_snapshots(before,
```

```
301     after}))
302 [padding]
303 [padding]
304 def _api_report(req, params):
305     text = build_markdown_report(STORE._state) # local-only
306     return text_response(200, text, content_type="text/markdown; charset=utf-
307 8")
308 [padding]
309 def _api_web_manifest(req, params):
310     return json_response(200, {"ok": True, "manifest":
311 build_manifest(WEB_ROOT)})
312 ROUTER.add("GET", "/api/health", _api_health)
313 ROUTER.add("POST", "/api/auth/login", _api_login)
314 ROUTER.add("GET", "/api/sites", _api_sites)
315 ROUTER.add("GET", "/api/plans", _api_plans)
316 ROUTER.add("GET", "/api/versions", _api_versions)
317 ROUTER.add("GET", "/api/plants", _api_plants)
318 ROUTER.add("POST", "/api/plants", _api_create_plant)
319 ROUTER.add("PUT", "/api/plants/{plantId}", _api_update_plant)
320 ROUTER.add("POST", "/api/plants/{plantId}/delete", _api_delete_plant)
321 ROUTER.add("POST", "/api/plants/import", _api_import_plants)
322 ROUTER.add("GET", "/api/materials", _api_materials)
323 ROUTER.add("POST", "/api/materials", _api_create_material)
324 ROUTER.add("PUT", "/api/materials/{materialId}", _api_update_material)
325 ROUTER.add("POST", "/api/materials/{materialId}/delete",
326 _api_delete_material)
327 ROUTER.add("GET", "/api/lights", _api_lights)
328 ROUTER.add("POST", "/api/lights", _api_create_light)
329 ROUTER.add("PUT", "/api/lights/{lightId}", _api_update_light)
330 ROUTER.add("POST", "/api/lights/{lightId}/delete", _api_delete_light)
331 ROUTER.add("GET", "/api/issues", _api_issues)
332 ROUTER.add("POST", "/api/versions/{versionId}/issues", _api_create_issue)
333 ROUTER.add("PUT", "/api/issues/{issueId}", _api_update_issue)
334 ROUTER.add("GET", "/api/pavings", _api_pavings)
335 ROUTER.add("POST", "/api/plans/{planId}/pavings", _api_create_paving)
336 ROUTER.add("GET", "/api/export", _api_export)
337 ROUTER.add("GET", "/api/versions/diff", _api_diff)
338 ROUTER.add("GET", "/api/report", _api_report)
339 ROUTER.add("GET", "/api/web/manifest", _api_web_manifest)
340 [padding]
341 [padding]
342 def main() -> None:
343     port = int(os.environ.get("APP_PORT") or "8010")
344     server = ThreadingHTTPServer(("127.0.0.1", port), Handler)
345     server.serve_forever()
346 [padding]
347 [padding]
348 if __name__ == "__main__":
349     main()
350 # apps/api/domain/router.py
```

```
351 # -*- coding: utf-8 -*-
352 from __future__ import annotations
353 [padding]
354 import json
355 from dataclasses import dataclass
356 from typing import Any, Callable
357 from urllib.parse import parse_qs, urlparse
358 [padding]
359 from .errors import ApiError, BadRequest, NotFound
360 [padding]
361 [padding]
362 @dataclass(frozen=True)
363 class Request:
364     method: str
365     path: str
366     query: dict[str, list[str]]
367     headers: dict[str, str]
368     body: bytes
369 [padding]
370     def json(self) -> dict[str, Any]:
371         if not self.body:
372             return {}
373         try:
374             return json.loads(self.body.decode("utf-8"))
375         except Exception as exc:
376             raise BadRequest("invalid_json") from exc
377 [padding]
378     def form(self) -> dict[str, str]:
379         raw = parse_qs(self.body.decode("utf-8", errors="ignore"))
380         return {k: (v[0] if v else "") for k, v in raw.items()}
381 [padding]
382 [padding]
383 Response = tuple[int, dict[str, str], bytes]
384 HandlerFn = Callable[[Request, dict[str, str]], Response]
385 [padding]
386 [padding]
387 def json_response(status: int, payload: dict[str, Any]) -> Response:
388     data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
389     return status, {"Content-Type": "application/json; charset=utf-8",
390 "Content-Length": str(len(data))}, data
391 [padding]
392 [padding]
393 def text_response(status: int, text: str, *, content_type: str = "text/plain;
394 charset=utf-8") -> Response:
395     data = text.encode("utf-8")
396     return status, {"Content-Type": content_type, "Content-Length":
397 str(len(data))}, data
398 [padding]
399 [padding]
400 class Router:
```



```

401     def __init__(self) -> None:
402         self._routes: list[tuple[str, str, HandlerFn]] = []
403 [padding]
404     def add(self, method: str, pattern: str, handler: HandlerFn) -> None:
405         self._routes.append((method.upper(), pattern, handler))
406 [padding]
407     def match(self, req: Request) -> tuple[HandlerFn, dict[str, str]]:
408         for method, pattern, fn in self._routes:
409             if method != req.method.upper():
410                 continue
411             params = self._match_pattern(pattern, req.path)
412             if params is not None:
413                 return fn, params
414             raise NotFound()
415 [padding]
416     @staticmethod
417     def _match_pattern(pattern: str, path: str) -> dict[str, str] | None:
418         # Very small path param matcher: "/api/plans/{planId}/versions"
419         p_parts = [p for p in pattern.split("/") if p]
420         a_parts = [p for p in path.split("/") if p]
421         if len(p_parts) != len(a_parts):
422             return None
423         params: dict[str, str] = {}
424         for p, a in zip(p_parts, a_parts, strict=True):
425             if p.startswith("{") and p.endswith("}"):
426                 params[p[1:-1]] = a
427             elif p != a:
428                 return None
429         return params
430 [padding]
431 [padding]
432     def build_request(method: str, raw_path: str, headers: dict[str, str], body:
433 bytes) -> Request:
434         parsed = urlparse(raw_path)
435         return Request(method=method, path=parsed.path, query=
436 parse_qs(parsed.query), headers=headers, body=body)
437 [padding]
438 [padding]
439     def handle_api_error(exc: ApiError) -> Response:
440         return json_response(exc.status, {"ok": False, "error": exc.code,
441 "message": exc.message})
442 [padding]
443 # apps/api/routers/__init__.py
444 [padding]
445 # apps/api/domain/auth.py
446 # -*- coding: utf-8 -*-
447 from __future__ import annotations
448 [padding]
449 import secrets
450 from dataclasses import dataclass

```

```
451 [padding]
452 from .errors import Unauthorized
453 [padding]
454 [padding]
455 @dataclass(frozen=True)
456 class User:
457     username: str
458     role: str
459 [padding]
460 [padding]
461 _USERS = {
462     "admin": {"password": "admin123", "role": "admin"},
463     "designer": {"password": "design123", "role": "designer"},
464     "reviewer": {"password": "review123", "role": "reviewer"},
465 }
466 [padding]
467 [padding]
468 def authenticate(username: str, password: str) -> User:
469     record = _USERS.get(username)
470     if not record or record["password"] != password:
471         raise Unauthorized("invalid_credentials", code="invalid_credentials")
472 [padding]
473     return User(username=username, role=record["role"])
474 [padding]
475 [padding]
476 def issue_token(user: User) -> str:
477     # Local demo token. Server keeps no session; token only used for UI
478     hints.
479     return f"demo-{user.role}-{secrets.token_hex(8)}"
480 [padding]
481 # apps/api/domain/csv_tools.py
482 # -*- coding: utf-8 -*-
483 from __future__ import annotations
484 [padding]
485 import csv
486 import io
487 from dataclasses import dataclass
488 from typing import Iterable
489 [padding]
490 from .errors import BadRequest
491 [padding]
492 [padding]
493 def sniff_dialect(text: str) -> csv.Dialect:
494     sample = text[:4096]
495     try:
496         return csv.Sniffer().sniff(sample)
497     except Exception:
498         class _Default(csv.Dialect):
499             delimiter = ","
500             quotechar = ""
```

```

501         doublequote = True
502         skipinitialspace = False
503         lineterminator = "\n"
504         quoting = csv.QUOTE_MINIMAL
505     [padding]
506     return _Default()
507 [padding]
508 [padding]
509 def parse_csv(text: str) -> list[list[str]]:
510     if not isinstance(text, str) or not text.strip():
511         raise BadRequest("csv text is empty.")
512     dialect = sniff_dialect(text)
513     reader = csv.reader(io.StringIO(text), dialect=dialect)
514     rows = []
515     for row in reader:
516         if not row:
517             continue
518         if all(not str(cell).strip() for cell in row):
519             continue
520         rows.append([str(cell).strip() for cell in row])
521     if not rows:
522         raise BadRequest("csv contains no rows.")
523     return rows
524 [padding]
525 [padding]
526 def csv_to_text(rows: Iterable[Iterable[object]]) -> str:
527     buf = io.StringIO()
528     writer = csv.writer(buf, lineterminator="\n")
529     for row in rows:
530         writer.writerow(["" if v is None else str(v) for v in row])
531     return buf.getvalue()
532 [padding]
533 [padding]
534 @dataclass(frozen=True)
535 class Table:
536     header: list[str]
537     rows: list[list[str]]
538 [padding]
539     def col_index(self, name: str) -> int:
540         try:
541             return self.header.index(name)
542         except ValueError as exc:
543             raise BadRequest(f"missing column: {name}") from exc
544 [padding]
545     def get(self, row: list[str], name: str) -> str:
546         idx = self.col_index(name)
547         return row[idx] if idx < len(row) else ""
548 [padding]
549 [padding]
550 def as_table(rows: list[list[str]]) -> Table:

```

```
551     header = rows[0]
552     if len(header) < 2:
553         raise BadRequest("csv header too short.")
554     body = rows[1:]
555     return Table(header=header, rows=body)
556 [padding]
557 # apps/api/domain/errors.py
558 # -*- coding: utf-8 -*-
559 from __future__ import annotations
560 [padding]
561 [padding]
562 class ApiError(Exception):
563     def __init__(self, status: int, code: str, message: str) -> None:
564         super().__init__(message)
565         self.status = status
566         self.code = code
567         self.message = message
568 [padding]
569 [padding]
570 class BadRequest(ApiError):
571     def __init__(self, message: str, *, code: str = "bad_request") -> None:
572         super().__init__(400, code, message)
573 [padding]
574 [padding]
575 class Unauthorized(ApiError):
576     def __init__(self, message: str = "unauthorized", *, code: str =
577 "unauthorized") -> None:
578         super().__init__(401, code, message)
579 [padding]
580 [padding]
581 class NotFound(ApiError):
582     def __init__(self, message: str = "not_found", *, code: str =
583 "not_found") -> None:
584         super().__init__(404, code, message)
585 [padding]
586 [padding]
587 class Conflict(ApiError):
588     def __init__(self, message: str = "conflict", *, code: str = "conflict")
589 -> None:
590         super().__init__(409, code, message)
591 [padding]
592 # apps/api/domain/exporter.py
593 # -*- coding: utf-8 -*-
594 from __future__ import annotations
595 [padding]
596 from dataclasses import dataclass
597 from io import StringIO
598 [padding]
599 from .errors import BadRequest
600 from .store import ProjectStore
```

```

601 [padding]
602 [padding]
603 def _csv_escape(value: object) -> str:
604     s = "" if value is None else str(value)
605     s = s.replace("'", "''")
606     return f'"{s}"'
607 [padding]
608 [padding]
609 def to_csv(rows: list[list[object]]) -> str:
610     out = StringIO()
611     for row in rows:
612         out.write(",".join(_csv_escape(v) for v in row))
613         out.write("\n")
614     return out.getvalue()
615 [padding]
616 [padding]
617 @dataclass(frozen=True)
618 class ExportBundle:
619     plants_csv: str
620     materials_csv: str
621     lights_csv: str
622 [padding]
623 [padding]
624 def export_all(store: ProjectStore) -> ExportBundle:
625     plants = store.list_plants()
626     materials = store.list_materials()
627     lights = store.list_lights()
628     if not plants or not materials or not lights:
629         raise BadRequest("export requires non-empty
630 plants/materials/lights.")
631 [padding]
632     plants_rows: list[list[object]] = [["编号", "中文名", "类别",
633 "常绿/落叶", "冠幅范围(m)", "季相要点"]]
634     for p in plants:
635         evergreen = "常绿" if bool(p.get("evergreen")) else "落叶"
636         crown = f"{p.get('crownMinM')}-{p.get('crownMaxM')}"
637         plants_rows.append([p.get("code"), p.get("cnName"), p.get("category")
638 , evergreen, crown, p.get("seasonHint")])
639 [padding]
640     materials_rows: list[list[object]] = [["编号", "名称", "单位", "用途"]]
641     for m in materials:
642         materials_rows.append([m.get("code"), m.get("name"), m.get("unit"),
643 m.get("usage")])
644 [padding]
645     lights_rows: list[list[object]] = [["编号", "色温", "功率(W)", "数量"]]
646     for l in lights:
647         lights_rows.append([l.get("code"), l.get("cct"), l.get("watt"),
648 l.get("qty")])
649 [padding]
650     return ExportBundle(plants_csv=to_csv(plants_rows), materials_csv=

```

```
651 to_csv(materials_rows), lights_csv=to_csv(lights_rows))
652 [padding]
653 # apps/api/domain/geometry.py
654 # -*- coding: utf-8 -*-
655 from __future__ import annotations
656 [padding]
657 from dataclasses import dataclass
658 from math import hypot
659 [padding]
660 from .errors import BadRequest
661 [padding]
662 [padding]
663 @dataclass(frozen=True)
664 class Point:
665     x: float
666     y: float
667 [padding]
668 [padding]
669 def parse_point(obj: object, *, name: str = "point") -> Point:
670     if not isinstance(obj, dict):
671         raise BadRequest(f"{name} must be an object with x/y.")
672     x = obj.get("x")
673     y = obj.get("y")
674     if not isinstance(x, (int, float)) or not isinstance(y, (int, float)):
675         raise BadRequest(f"{name}.x and {name}.y must be numbers.")
676     return Point(float(x), float(y))
677 [padding]
678 [padding]
679 def polyline_length(points: list[Point]) -> float:
680     if len(points) < 2:
681         return 0.0
682     total = 0.0
683     for a, b in zip(points, points[1:], strict=False):
684         total += hypot(b.x - a.x, b.y - a.y)
685     return total
686 [padding]
687 [padding]
688 def polygon_area(points: list[Point]) -> float:
689     if len(points) < 3:
690         return 0.0
691     s = 0.0
692     for a, b in zip(points, points[1:] + [points[0]], strict=False):
693         s += a.x * b.y - b.x * a.y
694     return abs(s) * 0.5
695 [padding]
696 [padding]
697 def validate_polygon(points: list[Point]) -> None:
698     if len(points) < 3:
699         raise BadRequest("polygon requires >= 3 points.")
700     if polygon_area(points) <= 1e-9:
```

```
701         raise BadRequest("polygon area too small or degenerate.")
702     [padding]
703     # apps/api/domain/ids.py
704     # -*- coding: utf-8 -*-
705     from __future__ import annotations
706     [padding]
707     import secrets
708     import time
709     [padding]
710     [padding]
711     def new_id(prefix: str) -> str:
712         # Stable-enough for local use; easy to read in exported CSV.
713         ts = int(time.time() * 1000)
714         rnd = secrets.token_hex(3)
715         return f"{prefix}-{ts:x}-{rnd}"
716     [padding]
717     # apps/api/domain/reporting.py
718     # -*- coding: utf-8 -*-
719     from __future__ import annotations
720     [padding]
721     """
722     Reporting utilities for local project review.
723     [padding]
724     This module focuses on deterministic, explainable summaries that can be used
725     in:
726     - internal QA before publishing a plan version;
727     - exporting a "review pack" for stakeholders;
728     - consistency checks between exported lists and on-screen data.
729     [padding]
730     The functions are intentionally written without external dependencies to
731     keep the
732     runtime self-contained.
733     """
734     [padding]
735     from dataclasses import dataclass
736     from datetime import datetime
737     from typing import Any, Iterable
738     [padding]
739     from .errors import BadRequest
740     [padding]
741     [padding]
742     def _now() -> str:
743         return datetime.now().replace(microsecond=0).isoformat(sep=" ")
744     [padding]
745     [padding]
746     def _as_list(value: object) -> list:
747         if value is None:
748             return []
749         if isinstance(value, list):
750             return value
```

```
751         raise BadRequest("expected list")
752     [padding]
753     [padding]
754     def _as_dict(value: object) -> dict:
755         if value is None:
756             return {}
757         if isinstance(value, dict):
758             return value
759         raise BadRequest("expected object")
760     [padding]
761     [padding]
762     def _norm_text(s: object) -> str:
763         return (" " if s is None else str(s)).strip()
764     [padding]
765     [padding]
766     def _safe_float(v: object, *, default: float = 0.0) -> float:
767         if v is None:
768             return default
769         if isinstance(v, (int, float)):
770             return float(v)
771         try:
772             return float(str(v).strip())
773         except Exception:
774             return default
775     [padding]
776     [padding]
777     def _safe_int(v: object, *, default: int = 0) -> int:
778         if v is None:
779             return default
780         if isinstance(v, bool):
781             return int(v)
782         if isinstance(v, (int, float)):
783             return int(v)
784         try:
785             return int(float(str(v).strip()))
786         except Exception:
787             return default
788     [padding]
789     [padding]
790     def _pct(part: int, total: int) -> str:
791         if total <= 0:
792             return "0%"
793         return f"{round(part * 100.0 / total)}%"
794     [padding]
795     [padding]
796     @dataclass(frozen=True)
797     class Kpi:
798         sites: int
799         plans: int
800         versions: int
```



```
801     plants: int
802     materials: int
803     lights: int
804     issues: int
805     pavings: int
806 [padding]
807     def to_dict(self) -> dict[str, int]:
808         return {
809             "sites": self.sites,
810             "plans": self.plans,
811             "versions": self.versions,
812             "plants": self.plants,
813             "materials": self.materials,
814             "lights": self.lights,
815             "issues": self.issues,
816             "pavings": self.pavings,
817         }
818 [padding]
819 [padding]
820     def compute_kpi(state: dict[str, Any]) -> Kpi:
821         return Kpi(
822             sites=len(_as_list(state.get("sites"))),
823             plans=len(_as_list(state.get("plans"))),
824             versions=len(_as_list(state.get("versions"))),
825             plants=len(_as_list(state.get("plants"))),
826             materials=len(_as_list(state.get("materials"))),
827             lights=len(_as_list(state.get("lights"))),
828             issues=len(_as_list(state.get("issues"))),
829             pavings=len(_as_list(state.get("pavings"))),
830         )
831 [padding]
832 [padding]
833     def group_by(items: Iterable[dict[str, Any]], key: str) -> dict[str,
834 list[dict[str, Any]]]:
835         out: dict[str, list[dict[str, Any]]] = {}
836         for it in items:
837             k = _norm_text(it.get(key))
838             out.setdefault(k, []).append(it)
839         return out
840 [padding]
841 [padding]
842     def summarize_plants(plants: list[dict[str, Any]]) -> dict[str, Any]:
843         by_category = group_by(plants, "category")
844         evergreen_count = 0
845         for p in plants:
846             if bool(p.get("evergreen")):
847                 evergreen_count += 1
848         summary = {
849             "total": len(plants),
850             "evergreen": evergreen_count,
```

```

851         "deciduous": len(plants) - evergreen_count,
852         "evergreenRatio": _pct(evergreen_count, len(plants)),
853         "byCategory": {k or "(未分类)": len(v) for k, v in
854 sorted(by_category.items(), key=lambda kv: kv[0])},
855     }
856     crown_ranges = []
857     for p in plants:
858         mn = _safe_float(p.get("crownMinM"))
859         mx = _safe_float(p.get("crownMaxM"))
860         if mn > 0 and mx > 0 and mx >= mn:
861             crown_ranges.append((mn, mx))
862     if crown_ranges:
863         summary["crownMinMin"] = min(mn for mn, _ in crown_ranges)
864         summary["crownMaxMax"] = max(mx for _, mx in crown_ranges)
865     return summary
866 [padding]
867 [padding]
868 def summarize_materials(materials: list[dict[str, Any]]) -> dict[str, Any]:
869     by_unit = group_by(materials, "unit")
870     return {
871         "total": len(materials),
872         "byUnit": {k or "(未定义单位)": len(v) for k, v in
873 sorted(by_unit.items(), key=lambda kv: kv[0])},
874     }
875 [padding]
876 [padding]
877 def summarize_lights(lights: list[dict[str, Any]]) -> dict[str, Any]:
878     total_qty = sum(_safe_int(l.get("qty")) for l in lights)
879     by_cct = group_by(lights, "cct")
880     return {
881         "total": len(lights),
882         "totalQty": total_qty,
883         "byCct": {k or "(未定义色温)": len(v) for k, v in
884 sorted(by_cct.items(), key=lambda kv: kv[0])},
885     }
886 [padding]
887 [padding]
888 def summarize_issues(issues: list[dict[str, Any]]) -> dict[str, Any]:
889     by_status = group_by(issues, "status")
890     by_tag = group_by(issues, "tag")
891     by_priority = group_by(issues, "priority")
892     return {
893         "total": len(issues),
894         "byStatus": {k or "(未定义状态)": len(v) for k, v in
895 sorted(by_status.items(), key=lambda kv: kv[0])},
896         "byTag": {k or "(未定义标签)": len(v) for k, v in
897 sorted(by_tag.items(), key=lambda kv: kv[0])},
898         "byPriority": {k or "(未定义优先级)": len(v) for k, v in
899 sorted(by_priority.items(), key=lambda kv: kv[0])},
900     }

```

```

901 [padding]
902 [padding]
903 @dataclass(frozen=True)
904 class ConsistencyIssue:
905     level: str # info/warn/error
906     topic: str
907     message: str
908 [padding]
909     def to_dict(self) -> dict[str, str]:
910         return {"level": self.level, "topic": self.topic, "message":
911 self.message}
912 [padding]
913 [padding]
914 def check_code_uniqueness(items: list[dict[str, Any]], *, topic: str) ->
915 list[ConsistencyIssue]:
916     seen: set[str] = set()
917     issues: list[ConsistencyIssue] = []
918     for it in items:
919         code = _norm_text(it.get("code")).upper()
920         if not code:
921             issues.append(ConsistencyIssue("error", topic, "missing code"))
922             continue
923         if code in seen:
924             issues.append(ConsistencyIssue("error", topic, f"duplicate code:
925 {code}"))
926             continue
927         seen.add(code)
928     return issues
929 [padding]
930 [padding]
931 def check_issue_flow(issues: list[dict[str, Any]]) -> list[ConsistencyIssue]
932 :
933     allowed = {"新建", "处理中", "已解决", "已关闭"}
934     out: list[ConsistencyIssue] = []
935     for it in issues:
936         status = _norm_text(it.get("status"))
937         if status and status not in allowed:
938             out.append(ConsistencyIssue("warn", "issues", f"unknown status:
939 {status}"))
940     return out
941 [padding]
942 [padding]
943 def check_theme_keywords(plans: list[dict[str, Any]]) ->
944 list[ConsistencyIssue]:
945     out: list[ConsistencyIssue] = []
946     for p in plans:
947         theme = _norm_text(p.get("theme"))
948         if not theme:
949             out.append(ConsistencyIssue("warn", "plans", "theme is empty"))
950         elif len(theme) > 20:

```

```

951         out.append(ConsistencyIssue("info", "plans", "theme text is long;
952 consider shorter label"))
953     return out
954 [padding]
955 [padding]
956 def run_consistency_checks(state: dict[str, Any]) -> list[ConsistencyIssue]:
957     plants = _as_list(state.get("plants"))
958     materials = _as_list(state.get("materials"))
959     lights = _as_list(state.get("lights"))
960     issues = _as_list(state.get("issues"))
961     plans = _as_list(state.get("plans"))
962 [padding]
963     results: list[ConsistencyIssue] = []
964     results.extend(check_code_uniqueness(plants, topic="plants"))
965     results.extend(check_code_uniqueness(materials, topic="materials"))
966     results.extend(check_code_uniqueness(lights, topic="lights"))
967     results.extend(check_issue_flow(issues))
968     results.extend(check_theme_keywords(plans))
969     return results
970 [padding]
971 [padding]
972 def build_markdown_report(state: dict[str, Any]) -> str:
973     kpi = compute_kpi(state)
974     plants = _as_list(state.get("plants"))
975     materials = _as_list(state.get("materials"))
976     lights = _as_list(state.get("lights"))
977     issues = _as_list(state.get("issues"))
978     checks = run_consistency_checks(state)
979 [padding]
980     p_sum = summarize_plants(plants)
981     m_sum = summarize_materials(materials)
982     l_sum = summarize_lights(lights)
983     i_sum = summarize_issues(issues)
984 [padding]
985     lines: list[str] = []
986     lines.append(f"# 项目自检报告")
987     lines.append("")
988     lines.append(f"- 生成时间: { _now() }")
989     lines.append("")
990     lines.append("## 1 关键指标")
991     lines.append("")
992     for k, v in kpi.to_dict().items():
993         lines.append(f"- {k}: {v}")
994     lines.append("")
995     lines.append("## 2 植物库概览")
996     lines.append("")
997     lines.append(f"- 总数: {p_sum['total']}")
998     lines.append(f"- 常绿/落叶: {p_sum['evergreen']}/{p_sum['deciduous']}")
999     (常绿占比 {p_sum['evergreenRatio']} ) )
1000     if "crownMinMin" in p_sum and "crownMaxMax" in p_sum:

```

```
1001         lines.append(f'- 冠幅范围覆盖: {p_sum['crownMinMin']}m ~
1002 {p_sum['crownMaxMax']}m")
1003     lines.append("- 类别分布: ")
1004     for k, v in p_sum["byCategory"].items():
1005         lines.append(f' - {k}: {v}')
1006     lines.append("")
1007     lines.append("## 3 材料字典概览")
1008     lines.append("")
1009     lines.append(f'- 总数: {m_sum['total']}')
1010     lines.append("- 单位分布: ")
1011     for k, v in m_sum["byUnit"].items():
1012         lines.append(f' - {k}: {v}')
1013     lines.append("")
1014     lines.append("## 4 灯具清单概览")
1015     lines.append("")
1016     lines.append(f'- 条目数: {l_sum['total']}')
1017     lines.append(f'- 数量合计: {l_sum['totalQty']}')
1018     lines.append("- 色温分布: ")
1019     for k, v in l_sum["byCct"].items():
1020         lines.append(f' - {k}: {v}')
1021     lines.append("")
1022     lines.append("## 5 评审事项概览")
1023     lines.append("")
1024     lines.append(f'- 总数: {i_sum['total']}')
1025     lines.append("- 状态分布: ")
1026     for k, v in i_sum["byStatus"].items():
1027         lines.append(f' - {k}: {v}')
1028     lines.append("- 标签分布: ")
1029     for k, v in i_sum["byTag"].items():
1030         lines.append(f' - {k}: {v}')
1031     lines.append("- 优先级分布: ")
1032     for k, v in i_sum["byPriority"].items():
1033         lines.append(f' - {k}: {v}')
1034     lines.append("")
1035     lines.append("## 6 一致性检查")
1036     lines.append("")
1037     if not checks:
1038         lines.append("- 未发现问题。")
1039     else:
1040         for it in checks:
1041             lines.append(f'- [{it.level}] {it.topic}: {it.message}')
1042     lines.append("")
1043     lines.append("## 7 建议")
1044     lines.append("")
1045     lines.append("- 发布版本前, 优先关闭高优先级事项, 确保变更摘要覆盖关键对
1046 象。")
1047     lines.append("- 清单导出字段应与施工深化口径一致, 必要时补充规格/工艺/等
1048 级字段。")
1049     lines.append("- 对于涉及安全与无障碍的调整, 建议在评审事项中明确原因与验
1050 收标准。")
```

```
1051     lines.append("")
1052     return "\n".join(lines) + "\n"
1053 [padding]
1054 # apps/api/domain/snapshot.py
1055 # -*- coding: utf-8 -*-
1056 from __future__ import annotations
1057 [padding]
1058 """
1059 Snapshot utilities.
1060 [padding]
1061 Rationale:
1062 - A plan "version" should be reviewable after data changes.
1063 - A snapshot is a plain JSON object that contains the relevant collections.
1064 - This module provides deterministic snapshot generation and validation
1065 helpers.
1066 """
1067 [padding]
1068 from dataclasses import dataclass
1069 from typing import Any
1070 [padding]
1071 from .errors import BadRequest
1072 [padding]
1073 [padding]
1074 @dataclass(frozen=True)
1075 class Snapshot:
1076     season: str
1077     plants: list[dict[str, Any]]
1078     materials: list[dict[str, Any]]
1079     lights: list[dict[str, Any]]
1080     issues: list[dict[str, Any]]
1081     pavings: list[dict[str, Any]]
1082 [padding]
1083     def to_dict(self) -> dict[str, Any]:
1084         return {
1085             "season": self.season,
1086             "plants": list(self.plants),
1087             "materials": list(self.materials),
1088             "lights": list(self.lights),
1089             "issues": list(self.issues),
1090             "pavings": list(self.pavings),
1091         }
1092 [padding]
1093 [padding]
1094     def _as_list(value: object, *, name: str) -> list:
1095         if value is None:
1096             return []
1097         if isinstance(value, list):
1098             return value
1099         raise BadRequest(f"{name} must be list")
1100 [padding]
```

```

1101 [padding]
1102 def _as_str(value: object, *, name: str, default: str = "") -> str:
1103     if value is None:
1104         return default
1105     s = str(value).strip()
1106     return s
1107 [padding]
1108 [padding]
1109 def build_snapshot(
1110     *,
1111     season: str,
1112     plants: list[dict[str, Any]],
1113     materials: list[dict[str, Any]],
1114     lights: list[dict[str, Any]],
1115     issues: list[dict[str, Any]],
1116     pavings: list[dict[str, Any]],
1117 ) -> Snapshot:
1118     season = _as_str(season, name="season", default="春") or "春"
1119     return Snapshot(
1120         season=season,
1121         plants=list(plants),
1122         materials=list(materials),
1123         lights=list(lights),
1124         issues=list(issues),
1125         pavings=list(pavings),
1126     )
1127 [padding]
1128 [padding]
1129 def validate_snapshot(payload: dict[str, Any]) -> None:
1130     if not isinstance(payload, dict):
1131         raise BadRequest("snapshot must be object")
1132     season = _as_str(payload.get("season"), name="season", default="")
1133     if season and season not in {"春", "夏", "秋", "冬"}:
1134         raise BadRequest("season must be one of 春/夏/秋/冬")
1135 [padding]
1136     for key in ("plants", "materials", "lights", "issues", "pavings"):
1137         _as_list(payload.get(key), name=key)
1138 [padding]
1139 [padding]
1140 def summarize_snapshot(payload: dict[str, Any]) -> dict[str, Any]:
1141     validate_snapshot(payload)
1142     season = _as_str(payload.get("season"), name="season", default="春") or
1143     "春"
1144     plants = _as_list(payload.get("plants"), name="plants")
1145     materials = _as_list(payload.get("materials"), name="materials")
1146     lights = _as_list(payload.get("lights"), name="lights")
1147     issues = _as_list(payload.get("issues"), name="issues")
1148     pavings = _as_list(payload.get("pavings"), name="pavings")
1149     return {
1150         "season": season,

```

```

1151         "counts": {
1152             "plants": len(plants),
1153             "materials": len(materials),
1154             "lights": len(lights),
1155             "issues": len(issues),
1156             "pavings": len(pavings),
1157         },
1158         "checks": {
1159             "hasPlants": len(plants) > 0,
1160             "hasMaterials": len(materials) > 0,
1161             "hasLights": len(lights) > 0,
1162         },
1163     }
1164 [padding]
1165 # apps/api/domain/static_files.py
1166 # -*- coding: utf-8 -*-
1167 from __future__ import annotations
1168 [padding]
1169 """
1170 Static file helper used by the built-in HTTP server.
1171 [padding]
1172 Goals:
1173 - keep path traversal safe;
1174 - provide consistent content-type;
1175 - support simple cache headers to reduce reload overhead during review;
1176 - avoid external dependencies.
1177 """
1178 [padding]
1179 import hashlib
1180 import mimetypes
1181 import os
1182 from dataclasses import dataclass
1183 from pathlib import Path
1184 from typing import Iterable
1185 [padding]
1186 from .errors import NotFound
1187 [padding]
1188 [padding]
1189 TEXT_PREFIXES = ("text/",)
1190 EXTRA_TEXT_TYPES = {
1191     ".js": "text/javascript",
1192     ".mjs": "text/javascript",
1193     ".css": "text/css",
1194     ".html": "text/html",
1195     ".json": "application/json",
1196     ".md": "text/markdown",
1197     ".svg": "image/svg+xml",
1198 }
1199 [padding]
1200 [padding]

```



```

1201 def safe_join(root: Path, rel: str) -> Path | None:
1202     rel = rel.lstrip("/").replace("/", os.sep)
1203     target = (root / rel).resolve()
1204     try:
1205         target.relative_to(root.resolve())
1206     except ValueError:
1207         return None
1208     return target
1209 [padding]
1210 [padding]
1211 def guess_content_type(path: Path) -> str:
1212     ctype, _ = mimetypes.guess_type(str(path))
1213     if ctype:
1214         return ctype
1215     return EXTRA_TEXT_TYPES.get(path.suffix.lower(), "application/octet-
1216 stream")
1217 [padding]
1218 [padding]
1219 def is_text_type(content_type: str) -> bool:
1220     return any(content_type.startswith(p) for p in TEXT_PREFIXES) or
1221 content_type in EXTRA_TEXT_TYPES.values()
1222 [padding]
1223 [padding]
1224 def compute_etag(data: bytes) -> str:
1225     # Weak etag is fine for local use.
1226     digest = hashlib.sha1(data).hexdigest()
1227     return f'W/{digest}'
1228 [padding]
1229 [padding]
1230 @dataclass(frozen=True)
1231 class StaticResponse:
1232     status: int
1233     headers: dict[str, str]
1234     body: bytes
1235 [padding]
1236 [padding]
1237 def build_static_response(
1238     *,
1239     root: Path,
1240     request_path: str,
1241     fallback: str = "index.html",
1242     cache_seconds: int = 30,
1243 ) -> StaticResponse:
1244     # Default "/" to index.
1245     path = request_path
1246     if path == "/" or path == "":
1247         path = "/" + fallback
1248     target = safe_join(root, path)
1249     if target is None or not target.exists() or not target.is_file():
1250         # SPA fallback

```

```

1251         target = root / fallback
1252         if not target.exists():
1253             raise NotFound("static file not found")
1254     [padding]
1255     data = target.read_bytes()
1256     ctype = guess_content_type(target)
1257     headers = {
1258         "Content-Type": f"{ctype}; charset=utf-8" if is_text_type(ctype)
1259     else ctype,
1260         "Content-Length": str(len(data)),
1261         "Cache-Control": f"public, max-age={max(cache_seconds,0)}",
1262         "ETag": compute_etag(data),
1263     }
1264     return StaticResponse(status=200, headers=headers, body=data)
1265 [padding]
1266 [padding]
1267 def iter_files(root: Path, *, suffixes: Iterable[str] | None = None) ->
1268 list[Path]:
1269     out: list[Path] = []
1270     if not root.exists():
1271         return out
1272     for p in sorted(root.rglob("*")):
1273         if not p.is_file():
1274             continue
1275         if suffixes is not None and p.suffix.lower() not in {s.lower() for s
1276 in suffixes}:
1277             continue
1278         out.append(p)
1279     return out
1280 [padding]
1281 [padding]
1282 def build_manifest(root: Path) -> dict[str, str]:
1283     """
1284     Return {relative_path: etag} for quick integrity checks.
1285     """
1286     manifest: dict[str, str] = {}
1287     for p in iter_files(root):
1288         rel = str(p.relative_to(root)).replace("\\", "/")
1289         try:
1290             manifest[rel] = compute_etag(p.read_bytes())
1291         except Exception:
1292             manifest[rel] = "W/^unreadable\"
1293     return manifest
1294 [padding]
1295 # apps/api/domain/store.py
1296 # -*- coding: utf-8 -*-
1297 from __future__ import annotations
1298 [padding]
1299 import json
1300 from dataclasses import dataclass, field

```

```
1301 from datetime import datetime
1302 from pathlib import Path
1303 from typing import Any
1304 [padding]
1305 from .errors import BadRequest, Conflict, NotFound
1306 from .geometry import Point, parse_point, polygon_area, validate_polygon
1307 from .ids import new_id
1308 from .validators import LightInput, MaterialInput, PlantInput,
1309 parse_light_input, parse_material_input, parse_plant_input, validate_code
1310 [padding]
1311 [padding]
1312 def _now_iso() -> str:
1313     return datetime.now().replace(microsecond=0).isoformat(sep=" ")
1314 [padding]
1315 [padding]
1316 def _read_json(path: Path, *, default: Any) -> Any:
1317     if not path.exists():
1318         return default
1319     return json.loads(path.read_text(encoding="utf-8"))
1320 [padding]
1321 [padding]
1322 def _write_json(path: Path, payload: Any) -> None:
1323     path.parent.mkdir(parents=True, exist_ok=True)
1324     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
1325 encoding="utf-8")
1326 [padding]
1327 [padding]
1328 @dataclass
1329 class Site:
1330     id: str
1331     name: str
1332     location: str
1333     scale_meter_per_px: float
1334     north_angle_deg: float
1335     created_at: str
1336 [padding]
1337 def to_dict(self) -> dict[str, Any]:
1338     return {
1339         "id": self.id,
1340         "name": self.name,
1341         "location": self.location,
1342         "scaleMeterPerPx": self.scale_meter_per_px,
1343         "northAngleDeg": self.north_angle_deg,
1344         "createdAt": self.created_at,
1345     }
1346 [padding]
1347 [padding]
1348 @dataclass
1349 class Plan:
1350     id: str
```

```
1351     site_id: str
1352     name: str
1353     theme: str
1354     created_by: str
1355     created_at: str
1356 [padding]
1357     def to_dict(self) -> dict[str, Any]:
1358         return {
1359             "id": self.id,
1360             "siteId": self.site_id,
1361             "name": self.name,
1362             "theme": self.theme,
1363             "createdBy": self.created_by,
1364             "createdAt": self.created_at,
1365         }
1366 [padding]
1367 [padding]
1368 @dataclass
1369 class PlanVersion:
1370     id: str
1371     plan_id: str
1372     version: str
1373     change_log: str
1374     created_at: str
1375     snapshot: dict[str, Any] = field(default_factory=dict)
1376 [padding]
1377     def to_dict(self) -> dict[str, Any]:
1378         return {
1379             "id": self.id,
1380             "planId": self.plan_id,
1381             "version": self.version,
1382             "changeLog": self.change_log,
1383             "createdAt": self.created_at,
1384             "snapshot": self.snapshot,
1385         }
1386 [padding]
1387 [padding]
1388 @dataclass
1389 class PlantCatalogItem:
1390     id: str
1391     code: str
1392     cn_name: str
1393     category: str
1394     evergreen: bool
1395     crown_min_m: float
1396     crown_max_m: float
1397     season_hint: str
1398 [padding]
1399     def to_dict(self) -> dict[str, Any]:
1400         return {
```

```
1401         "id": self.id,
1402         "code": self.code,
1403         "cnName": self.cn_name,
1404         "category": self.category,
1405         "evergreen": self.evergreen,
1406         "crownMinM": self.crown_min_m,
1407         "crownMaxM": self.crown_max_m,
1408         "seasonHint": self.season_hint,
1409     }
1410 [padding]
1411 [padding]
1412 @dataclass
1413 class MaterialItem:
1414     id: str
1415     code: str
1416     name: str
1417     unit: str
1418     usage: str
1419 [padding]
1420     def to_dict(self) -> dict[str, Any]:
1421         return {"id": self.id, "code": self.code, "name": self.name, "unit":
1422 self.unit, "usage": self.usage}
1423 [padding]
1424 [padding]
1425 @dataclass
1426 class LightItem:
1427     id: str
1428     code: str
1429     cct: str
1430     watt: int
1431     qty: int
1432 [padding]
1433     def to_dict(self) -> dict[str, Any]:
1434         return {"id": self.id, "code": self.code, "cct": self.cct, "watt":
1435 self.watt, "qty": self.qty}
1436 [padding]
1437 [padding]
1438 @dataclass
1439 class Issue:
1440     id: str
1441     version_id: str
1442     title: str
1443     tag: str
1444     priority: str
1445     status: str
1446     created_at: str
1447 [padding]
1448     def to_dict(self) -> dict[str, Any]:
1449         return {
1450             "id": self.id,
```

```

1451         "versionId": self.version_id,
1452         "title": self.title,
1453         "tag": self.tag,
1454         "priority": self.priority,
1455         "status": self.status,
1456         "createdAt": self.created_at,
1457     }
1458 [padding]
1459 [padding]
1460 @dataclass
1461 class PavingSurface:
1462     id: str
1463     plan_id: str
1464     material_code: str
1465     points: list[Point]
1466 [padding]
1467     def area_m2(self, *, meter_per_px: float) -> float:
1468         return polygon_area(self.points) * (meter_per_px**2)
1469 [padding]
1470     def to_dict(self) -> dict[str, Any]:
1471         return {
1472             "id": self.id,
1473             "planId": self.plan_id,
1474             "materialCode": self.material_code,
1475             "points": [{"x": p.x, "y": p.y} for p in self.points],
1476         }
1477 [padding]
1478 [padding]
1479 class ProjectStore:
1480     def __init__(self, base_dir: Path) -> None:
1481         self.base_dir = base_dir
1482         self.data_dir = base_dir / "apps" / "api" / "data"
1483         self.state_path = self.data_dir / "project_state.json"
1484         self._state = _read_json(self.state_path, default={})
1485         self._ensure_seed()
1486 [padding]
1487     def _ensure_seed(self) -> None:
1488         if self._state.get("seeded"):
1489             return
1490         site_id = new_id("S")
1491         plan_id = new_id("PLN")
1492         version_id = new_id("V")
1493         self._state = {
1494             "seeded": True,
1495             "sites": [
1496                 Site(
1497                     id=site_id,
1498                     name="滨水公园示范段",
1499                     location="华东 · 城市新区",
1500                     scale_meter_per_px=0.05,

```

```
1501         north_angle_deg=0.0,
1502         "latin": "Ophiopogon japonicus",
1503         "category": "地被",
1504         "evergreen": true,
1505         "crownMinM": 0.15,
1506         "crownMaxM": 0.25,
1507         "flowering": "",
1508         "seasonHint": "常绿耐阴，适合林下与边界收口。",
1509         "palette": { "春": "#4f7f55", "夏": "#2f6f4e", "秋": "#557a4a", "冬":
1510 "#2a4a33" },
1511     },
1512 [padding]
1513 {
1514     "code": "PL-0021",
1515     "cnName": "示例植物 021（滨水乔木）",
1516     "latin": "",
1517     "category": "乔木",
1518     "evergreen": false,
1519     "crownMinM": 4,
1520     "crownMaxM": 7,
1521     "flowering": "",
1522     "seasonHint": "用于滨水带背景林的示例条目。",
1523     "palette": { "春": "#6d8f58", "夏": "#4b7447", "秋": "#a38743", "冬":
1524 "#6a5a52" },
1525     },
1526     {
1527         "code": "PL-0022",
1528         "cnName": "示例植物 022（观花小乔）",
1529         "latin": "",
1530         "category": "乔木",
1531         "evergreen": false,
1532         "crownMinM": 3,
1533         "crownMaxM": 5,
1534         "flowering": "3-5 月",
1535         "seasonHint": "用于花径与节点的示例条目。",
1536         "palette": { "春": "#e7b2c7", "夏": "#4d7a4b", "秋": "#8e6a56", "冬":
1537 "#6b5a53" },
1538     },
1539     {
1540         "code": "PL-0023",
1541         "cnName": "示例植物 023（常绿灌木）",
1542         "latin": "",
1543         "category": "灌木",
1544         "evergreen": true,
1545         "crownMinM": 0.8,
1546         "crownMaxM": 1.6,
1547         "flowering": "",
1548         "seasonHint": "用于道路边界与基础绿篱的示例条目。",
1549         "palette": { "春": "#5f8a62", "夏": "#3e6f46", "秋": "#5c7a4e", "冬":
1550 "#2f4a34" }
```

```
1551     },
1552     {
1553         "code": "PL-0024",
1554         "cnName": "示例植物 024（彩叶灌木）",
1555         "latin": "",
1556         "category": "灌木",
1557         "evergreen": true,
1558         "crownMinM": 0.6,
1559         "crownMaxM": 1.4,
1560         "flowering": "",
1561         "seasonHint": "用于色块与节点强调的示例条目。",
1562         "palette": { "春": "#7a3b56", "夏": "#6b2f4a", "秋": "#7a3b56", "冬":
1563 "#4a2a3b" }
1564     },
1565     {
1566         "code": "PL-0025",
1567         "cnName": "示例植物 025（地被边界）",
1568         "latin": "",
1569         "category": "地被",
1570         "evergreen": true,
1571         "crownMinM": 0.12,
1572         "crownMaxM": 0.25,
1573         "flowering": "",
1574         "seasonHint": "用于铺装边界收口的示例条目。",
1575         "palette": { "春": "#4f7f55", "夏": "#2f6f4e", "秋": "#557a4a", "冬":
1576 "#2a4a33" }
1577     }
1578 ]
1579 }
1580 # apps/api/data/project_state.json
1581 {
1582     "seeded": true,
1583     "sites": [
1584         {
1585             "id": "S-19e6ea1e014-782e03",
1586             "name": "滨水公园示范段",
1587             "location": "华东 · 城市新区",
1588             "scaleMeterPerPx": 0.05,
1589             "northAngleDeg": 0.0,
1590             "createdAt": "2026-05-28 20:49:22"
1591         }
1592     ],
1593     "plans": [
1594         {
1595             "id": "PLN-19e6ea1e014-6b289d",
1596             "siteId": "S-19e6ea1e014-782e03",
1597             "name": "自然雅致方案",
1598             "theme": "自然雅致",
1599             "createdBy": "designer",
1600             "createdAt": "2026-05-28 20:49:22"
```



```
1601     }
1602   ],
1603   "versions": [
1604     {
1605       "id": "V-19e6eae014-973906",
1606       "planId": "PLN-19e6eae014-6b289d",
1607       "version": "V1.0.0",
1608       "changeLog": "初版：建立场地基准，整理植物与材料口径，形成评审事项清单
1609。",
1610       "createdAt": "2026-05-28 20:49:22",
1611       "snapshot": {
1612         "season": "春"
1613       }
1614     }
1615   ],
1616   "plants": [
1617     {
1618       "id": "P-19e6eae014-c0180c",
1619       "code": "P-001",
1620       "cnName": "香樟",
1621       "category": "乔木",
1622       "evergreen": true,
1623       "crownMinM": 6,
1624       "crownMaxM": 10,
1625       "seasonHint": "四季常绿，适合作为骨架乔木。"
1626     },
1627     {
1628       "id": "P-19e6eae014-e350e2",
1629       "code": "P-002",
1630       "cnName": "紫薇",
1631       "category": "乔木",
1632       "evergreen": false,
1633       "crownMinM": 3,
1634       "crownMaxM": 5,
1635       "seasonHint": "夏季花期明显，适合节点色彩强调。"
1636     },
1637     {
1638       "id": "P-19e6eae014-0fc535",
1639       "code": "P-003",
1640       "cnName": "红枫",
1641       "category": "乔木",
1642       "evergreen": false,
1643       "crownMinM": 4,
1644       "crownMaxM": 7,
1645       "seasonHint": "秋季红叶，宜与常绿灌木形成对比。"
1646     },
1647     {
1648       "id": "P-19e6eae014-7e5f98",
1649       "code": "P-004",
1650       "cnName": "海桐",
```

```
1651     "category": "灌木",
1652     "evergreen": true,
1653     "crownMinM": 0.8,
1654     "crownMaxM": 1.2,
1655     "seasonHint": "常绿耐修剪，可用于分隔与基础绿篱。"
1656   }
1657 ],
1658   "materials": [
1659     {
1660       "id": "M-19e6ea1e014-3f2c1f",
1661       "code": "M-01",
1662       "name": "透水砖（暖灰）",
1663       "unit": "m²",
1664       "usage": "主园路"
1665     },
1666     {
1667       "id": "M-19e6ea1e014-3fc23b",
1668       "code": "M-02",
1669       "name": "花岗岩火烧面",
1670       "unit": "m²",
1671       "usage": "入口广场"
1672     },
1673     {
1674       "id": "M-19e6ea1e014-594aa8",
1675       "code": "M-03",
1676       "name": "防腐木平台板",
1677       "unit": "m²",
1678       "usage": "亲水平台"
1679     }
1680   ],
1681   "lights": [
1682     {
1683       "id": "L-19e6ea1e014-140b2f",
1684       "code": "L-01",
1685       "cct": "3000K",
1686       "watt": 12,
1687       "qty": 18
1688     },
1689     {
1690       "id": "L-19e6ea1e014-819d37",
1691       "code": "L-02",
1692       "cct": "3500K",
1693       "watt": 18,
1694       "qty": 6
1695     }
1696   ],
1697   "issues": [
1698     {
1699       "id": "R-1001",
1700       "versionId": "V-19e6ea1e014-973906",
```

```
1701     "title": "入口动线转角偏急",
1702     "tag": "动线",
1703     "priority": "高",
1704     "status": "新建",
1705     "createdAt": "2026-05-28 20:49:22"
1706   },
1707   {
1708     "id": "R-1002",
1709     "versionId": "V-19e6ea1e014-973906",
1710     "title": "水体边缘需增加防滑提示",
1711     "tag": "水体",
1712     "priority": "中",
1713     "status": "处理中",
1714     "createdAt": "2026-05-28 20:49:22"
1715   }
1716 ],
1717 "pavings": [
1718   {
1719     "id": "PV-19e6ea1e014-369dda",
1720     "planId": "PLN-19e6ea1e014-6b289d",
1721     "materialCode": "M-01",
1722     "points": [
1723       {
1724         "x": 10,
1725         "y": 10
1726       },
1727       {
1728         "x": 210,
1729         "y": 15
1730       },
1731       {
1732         "x": 230,
1733         "y": 120
1734       },
1735       {
1736         "x": 15,
1737         "y": 140
1738       }
1739     ]
1740   }
1741 ]
1742 }
1743 # apps/web/admin.html
1744 <!doctype html>
1745 <html lang="zh-CN">
1746 <head>
1747   <meta charset="utf-8" />
1748   <meta name="viewport" content="width=device-width, initial-scale=1" />
1749   <title>系统管理 - 园林景观三维可视化设计平台</title>
1750   <meta http-equiv="refresh" content="0; url=./index.html#site" />
```

```
1751 </head>
1752 <body>
1753   <p>正在跳转到系统管理...</p>
1754 </body>
1755 </html>
1756 # apps/web/app.css
1757 :root{
1758   --bg:#f6f7f4;
1759   --panel:#ffffff;
1760   --ink:#1f2a24;
1761   --muted:#55635b;
1762   --accent:#2f6f4e;
1763   --accent2:#9db9a7;
1764   --border:#e2e7e2;
1765   --shadow:0 10px 30px rgba(18,24,20,.08);
1766   --radius:14px;
1767 }
1768 *{box-sizing:border-box}
1769 body{
1770   margin:0;
1771   font-family: "Microsoft YaHei", "PingFang SC", system-ui, -apple-system,
1772   Segoe UI, Arial, sans-serif;
1773   color:var(--ink);
1774   background:linear-gradient(180deg,#eef2ee 0%, var(--bg) 40%, #f9faf8 100%);
1775   [padding]
1776 }
1777 a{color:var(--accent); text-decoration:none}
1778 header{
1779   position:sticky; top:0; z-index:10;
1780   background:rgba(246,247,244,.86);
1781   backdrop-filter: blur(10px);
1782   border-bottom:1px solid var(--border);
1783 }
1784 .topbar{
1785   max-width:1200px; margin:0 auto;
1786   padding:14px 18px;
1787   display:flex; gap:12px; align-items:center; justify-content:space-between;
1788 }
1789 .brand{
1790   display:flex; gap:10px; align-items:center;
1791 }
1792 .logo{
1793   width:34px; height:34px; border-radius:10px;
1794   background: radial-gradient(circle at 30% 30%, #b8d6c4 0%, #2f6f4e 55%,
1795   #254a38 100%);
1796   box-shadow: 0 8px 20px rgba(47,111,78,.25);
1797 }
1798 .brand h1{
1799   font-size:16px; margin:0;
1800 }
```

```
1801 .brand .sub{
1802     font-size:12px; color:var(--muted);
1803 }
1804 .nav{
1805     display:flex; gap:10px; flex-wrap:wrap;
1806 }
1807 .nav a{
1808     padding:8px 10px;
1809     border-radius:10px;
1810     color:var(--muted);
1811 }
1812 .nav a.active{
1813     color:var(--ink);
1814     background:rgba(47,111,78,.10);
1815     border:1px solid rgba(47,111,78,.18);
1816 }
1817 .wrap{max-width:1200px; margin:0 auto; padding:18px}
1818 .grid{
1819     display:grid;
1820     grid-template-columns: 1fr;
1821     gap:14px;
1822 }
1823 @media (min-width: 980px){
1824     .grid{grid-template-columns: 340px 1fr}
1825 }
1826 .card{
1827     background:var(--panel);
1828     border:1px solid var(--border);
1829     border-radius:var(--radius);
1830     box-shadow:var(--shadow);
1831 }
1832 .card .hd{
1833     padding:14px 16px;
1834     border-bottom:1px solid var(--border);
1835     display:flex; align-items:center; justify-content:space-between; gap:10px;
1836 }
1837 .card .hd h2{font-size:14px; margin:0}
1838 .card .bd{padding:14px 16px}
1839 .kpi{
1840     display:grid; grid-template-columns:repeat(2,1fr); gap:10px;
1841 }
1842 .kpi .box{
1843     border:1px solid var(--border);
1844     border-radius:12px;
1845     padding:10px;
1846     background:linear-gradient(180deg,#fff 0%, #fbfcfb 100%);
1847 }
1848 .kpi .num{font-size:18px; font-weight:700}
1849 .kpi .lab{font-size:12px; color:var(--muted)}
1850 .btn{
```

```
1851     appearance:none;
1852     border:1px solid rgba(47,111,78,.25);
1853     background:rgba(47,111,78,.10);
1854     color:var(--ink);
1855     border-radius:12px;
1856     padding:9px 12px;
1857     cursor:pointer;
1858 }
1859 .btn.primary{
1860     background:linear-gradient(180deg,#3a815b 0%, #2f6f4e 100%);
1861     color:#fff;
1862     border-color:rgba(0,0,0,0);
1863 }
1864 .btn.danger{
1865     background:rgba(159,45,45,.08);
1866     border-color:rgba(159,45,45,.22);
1867 }
1868 .row{display:flex; gap:10px; align-items:center; flex-wrap:wrap}
1869 label{font-size:12px; color:var(--muted)}
1870 input,select,textarea{
1871     width:100%;
1872     padding:10px 12px;
1873     border:1px solid var(--border);
1874     border-radius:12px;
1875     outline:none;
1876     background:#fff;
1877 }
1878 textarea{min-height:90px; resize:vertical}
1879 .field{display:grid; gap:6px; margin-bottom:10px}
1880 .muted{color:var(--muted); font-size:12px}
1881 table{width:100%; border-collapse:collapse}
1882 th,td{padding:10px 8px; border-bottom:1px solid var(--border); font-
1883 size:13px; text-align:left}
1884 th{color:var(--muted); font-weight:600; background:#fbfcfb}
1885 .pill{
1886     display:inline-flex; align-items:center; gap:6px;
1887     padding:4px 10px; border-radius:999px;
1888     border:1px solid rgba(47,111,78,.20);
1889     background:rgba(47,111,78,.08);
1890     font-size:12px;
1891 }
1892 .view-title{
1893     display:flex; align-items:flex-start; justify-content:space-between;
1894     gap:12px;
1895 }
1896 .view-title h3{margin:0; font-size:16px}
1897 .view-title p{margin:4px 0 0; font-size:12px; color:var(--muted)}
1898 [padding]
1899 .modal-shell{
1900     position:fixed; inset:0; display:none;
```

```
1901     background:rgba(10,12,11,.32);
1902     z-index:50;
1903 }
1904 .modal-shell.modal{display:flex; align-items:center; justify-content:center;
1905 padding:18px}
1906 .modal{
1907     width:min(760px, 96vw);
1908     background:#fff;
1909     border-radius:18px;
1910     border:1px solid var(--border);
1911     box-shadow: 0 18px 40px rgba(0,0,0,.18);
1912     overflow:hidden;
1913 }
1914 .modal .mhd{
1915     padding:14px 16px;
1916     border-bottom:1px solid var(--border);
1917     display:flex; align-items:center; justify-content:space-between; gap:10px;
1918 }
1919 .modal .mhd strong{font-size:14px}
1920 .modal .mbd{padding:14px 16px}
1921 .modal .mft{
1922     padding:14px 16px;
1923     border-top:1px solid var(--border);
1924     display:flex; justify-content:flex-end; gap:10px;
1925 }
1926 [padding]
1927 .split{
1928     display:grid; grid-template-columns:1fr; gap:12px;
1929 }
1930 @media (min-width: 820px){
1931     .split{grid-template-columns: 1fr 1fr}
1932 }
1933 [padding]
1934 # apps/web/app.js
1935 const App = () => {
1936     const state = {
1937         token: localStorage.getItem("g3d_token") || "",
1938         role: localStorage.getItem("g3d_role") || "",
1939         season: "春",
1940         plants: [],
1941         materials: [],
1942         lights: [],
1943         issues: [],
1944         versions: [],
1945     };
1946 [padding]
1947     async function apiGet(path) {
1948         const headers = {};
1949         if (state.token) headers["Authorization"] = `Bearer ${state.token}`;
1950         const res = await fetch(path, { headers });
```

```
1951     const data = await res.json().catch(() => ({}));
1952     if (!res.ok || data.ok === false) throw new Error(data.error ||
1953 "api_failed");
1954     return data;
1955   }
1956 [padding]
1957   async function loadCoreData() {
1958     const [plants, materials, lights, issues, versions] = await
1959 Promise.all([
1960     apiGet("/api/plants"),
1961     apiGet("/api/materials"),
1962     apiGet("/api/lights"),
1963     apiGet("/api/issues"),
1964     apiGet("/api/versions"),
1965   ]);
1966   state.plants = plants.items || [];
1967   state.materials = materials.items || [];
1968   state.lights = lights.items || [];
1969   state.issues = issues.items || [];
1970   state.versions = versions.items || [];
1971   }
1972 [padding]
1973   const views = {
1974     home: {
1975       title: "方案总览",
1976       desc: "聚焦场地、版本、评审与清单输出。",
1977       render: () => `
1978         <div class="grid">
1979           <section class="card">
1980             <div class="hd"><h2>快速入口</h2><span class=
1981 "pill">自然雅致主题</span></div>
1982             <div class="bd">
1983               <div class="row">
1984                 <a class="btn primary" href="#site">进入场地</a>
1985                 <a class="btn" href="#plan">方案编辑</a>
1986                 <a class="btn" href="#review">评审中心</a>
1987                 <a class="btn" href="#export">清单导出</a>
1988               </div>
1989               <div style="height:10px"></div>
1990               <div class="muted">演示账号： admin/admin123（用于自动截图与门
1991 禁）。</div>
1992             </div>
1993           </section>
1994           <section class="card">
1995             <div class="hd"><h2>关键指标</h2><span class="muted">V1.0
1996 数据示例</span></div>
1997             <div class="bd">
1998               <div class="kpi">
1999                 <div class="box"><div class="num">1</div><div class=
2000 "lab">场地</div></div></div>
```



```
2001      <div class="box"><div class="num">3</div><div class=
2002 "lab">方案版本</div></div>
2003      <div class="box"><div class="num">${state.plants.length}
2004 </div><div class="lab">植物品种</div></div>
2005      <div class="box"><div class="num">${state.issues.length}
2006 </div><div class="lab">评审事项</div></div>
2007      </div>
2008      </div>
2009      </section>
2010      </div>
2011      `,
2012    },
2013    site: {
2014      title: "场地与底图",
2015      desc: "底图标定、北向与比例尺，作为三维场景的基准。",
2016      render: () => `
2017        <section class="card">
2018          <div class="hd"><h2>场地信息</h2><span class=
2019 "pill">基准：平面坐标</span></div>
2020          <div class="bd split">
2021            <div>
2022              <div class="field"><label>场地名称</label><input value=
2023 "滨水公园示范段" readonly /></div>
2024              <div class="field"><label>位置</label><input value="华东 ·
2025 城市新区" readonly /></div>
2026              <div class="field"><label>比例尺标定</label><input value="1px =
2027 0.05m（两点测距标定）" readonly /></div>
2028              <div class="muted">提示： V1.0 以快速可视化与清单闭环为主，地形
2029 与构件为轻量参数化对象。</div>
2030            </div>
2031            <div>
2032              <div class="field"><label>底图（示意）</label><div class=
2033 "card" style="border-style:dashed; box-shadow:none; padding:14px;">
2034                </div></div>
2035              </div>
2036            </div>
2037          </section>
2038          `,
2039          },
2040          plan: {
2041            title: "方案编辑",
2042            desc: "道路、铺装、水体、构筑物与季相预览。",
2043            render: () => `
2044              <section class="card">
2045                <div class="hd">
2046                  <h2>编辑面板</h2>
2047                  <div class="row">
2048                    <span class="pill">季相： ${state.season}</span>
2049                    <button class="btn" id="season-btn">切换季节</button>
2050                    <a class="btn" href="#planting">植物布置</a>
```

```

2051     <a class="btn" href="#materials">材质库</a>
2052   </div>
2053 </div>
2054 <div class="bd">
2055   <div class="view-title">
2056     <div>
2057       <h3>三维视窗（演示）</h3>
2058       <p>真实三维渲染在 V1.0 规划中采用浏览器
2059 WebGL；此处用“场景要素清单”替代，以便截图、文档与代码闭环。</p>
2060     </div>
2061     <span class="pill">版本： V1.0.0</span>
2062   </div>
2063   <div style="height:8px"></div>
2064   <table>
2065     <thead><tr><th>要素</th><th>参数</th><th>说明</th></tr></thead>
2066   >
2067     <tbody>
2068       <tr><td>道路</td><td>中心线+宽度
2069 3.5m</td><td>主园路，圆角过渡</td></tr>
2070       <tr><td>铺装面</td><td>透水砖（暖灰）</td><td>入口广场与节点
2071 铺装</td></tr>
2072       <tr><td>水体</td><td>水面高程 +
2073 0.2m</td><td>浅水景观与亲水平台</td></tr>
2074       <tr><td>灯光</td><td>3000K 暖光</td><td>夜景路径引导</td></tr>
2075     </tbody>
2076   </table>
2077 </div>
2078 </section>
2079 `
2080 bind: () => {
2081   const btn = document.getElementById("season-btn");
2082   if (!btn) return;
2083   btn.addEventListener("click", () => {
2084     const order = ["春", "夏", "秋", "冬"];
2085     const idx = order.indexOf(state.season);
2086     state.season = order[(idx + 1) % order.length];
2087     render();
2088   });
2089 },
2090 },
2091 },
2092 planting: {
2093   title: "植物库与布置",
2094   desc: "品种、规格与季相信息统一到清单中。",
2095   render: () => `
2096     <section class="card">
2097       <div class="hd">
2098         <h2>植物库（可评审）</h2>
2099       <div class="row">
2100         <button class="btn primary" data-action="create">新增</button>

```

```

2101     <button class="btn" data-action="edit">编辑</button>
2102     <button class="btn" data-action="view">查看</button>
2103     <button class="btn danger" data-action="delete">删除</button>
2104 </div>
2105 </div>
2106 <div class="bd">
2107     <div style="height:10px"></div>
2108     <table>
2109         <thead><tr><th>编号</th><th>中文名</th><th>类别</th><th>常绿/
2110 落叶</th><th>冠幅</th><th>季相要点</th></tr></thead>
2111         <tbody>
2112             ${
2113                 state.plants
2114                 .map((p) => {
2115                     const evergreen = p.evergreen ? "常绿" : "落叶";
2116                     const crown = `${p.crownMinM}-${p.crownMaxM}m`;
2117                     return `<tr><td>${p.code}</td><td>${p.cnName}<
2118 </td><td>${p.category}</td><td>${evergreen}</td><td>${crown}<
2119 </td><td>${p.seasonHint || ""}</td></tr>`;
2120                 })
2121                 .join("")
2122             }
2123         </tbody>
2124     </table>
2125 </div>
2126 </section>
2127 `,
2128 bind: () => {
2129     document.querySelectorAll("[data-action]").forEach((el) => {
2130         el.addEventListener("click", () =>
2131 openPlantModal(el.getAttribute("data-action"))));
2132     });
2133 },
2134 },
2135 materials: {
2136     title: "材料与铺装",
2137     desc: "把视觉效果与工程清单映射到同一份材料字典。",
2138     render: () => `
2139         <section class="card">
2140             <div class="hd"><h2>材料字典</h2><span class="pill">低饱和 ·
2141 暖灰</span></div>
2142             <div class="bd">
2143                 <table>
2144                     <thead><tr><th>编号</th><th>名称</th><th>计量单位</th><th>用途
2145 </th></tr></thead>
2146                     <tbody>
2147                         ${state.materials.map(m => `<tr><td>${m.code}<
2148 </td><td>${m.name}</td><td>${m.unit}</td><td>${m.usage}</td></tr>`).join("")}
2149 [padding]
2150                     </tbody>

```

```

2151         </table>
2152     </div>
2153 </section>
2154 `,
2155 },
2156 review: {
2157     title: "评审中心",
2158     desc: "意见基于截图流转，版本可追溯。",
2159     render: () => `
2160         <section class="card">
2161             <div class="hd"><h2>评审事项</h2><span class=
2162 "pill">版本： V1.0.0</span></div>
2163             <div class="bd">
2164                 <table>
2165                     <thead><tr><th>编号</th><th>标题</th><th>标签</th><th>优先级</
2166 th><th>状态</th></tr></thead>
2167                     <tbody>
2168                         ${state.issues.map(i => `<tr><td>${i.id}</td><td>${i.title}
2169 </td><td>${i.tag}</td><td>${i.priority}</td><td>${i.status}</td></tr>`)
2170 .join("")}
2171                     </tbody>
2172                 </table>
2173                 <div style="height:10px"></div>
2174             </div>
2175         </section>
2176     `,
2177 },
2178 export: {
2179     title: "清单导出",
2180     desc: "植物、材料与灯具清单以 CSV 输出，支撑交付与招采。",
2181     render: () => `
2182         <section class="card">
2183             <div class="hd"><h2>导出面板</h2><span class=
2184 "muted">示例导出</span></div>
2185             <div class="bd">
2186                 <div class="row">
2187                     <button class="btn primary" id="export-
2188 plants">导出植物清单</button>
2189                     <button class="btn" id="export-
2190 materials">导出材料清单</button>
2191                     <button class="btn" id="export-lights">导出灯具清单</button>
2192                 </div>
2193                 <div style="height:12px"></div>
2194                 <pre id="export-out" class="card" style="box-shadow:none;
2195 background:#fbfcfb; border-style:dashed; padding:12px; white-space:pre-wrap;
2196 margin:0;"></pre>
2197             </div>
2198         </section>
2199     `,
2200     bind: () => {

```

```

2201     const out = document.getElementById("export-out");
2202     const write = (txt) => { if(out) out.textContent = txt; };
2203     const csv = (rows) => rows.map(r => r.map(v => `${String(v)
2204 .replaceAll("'", "''")}`)).join(",").join("\n");
2205     document.getElementById("export-plants").addEventListener("click",
2206 () => {
2207         write(csv([[ "编号", "中文名", "类别", "常绿/落叶", "冠幅范围(m)",
2208 ...state.plants.map(p=>[p.code,p.cnName,p.category,(p.evergreen?
2209 "常绿":"落叶"),`${p.crownMinM}-${p.crownMaxM}` ]]]));
2210     });
2211     document.getElementById("export-materials")
2212 ?.addEventListener("click", () => {
2213         write(csv([[ "编号", "名称", "单位", "用途", ...state.materials.map(m=
2214 >[m.code,m.name,m.unit,m.usage]]]));
2215     });
2216     document.getElementById("export-lights").addEventListener("click",
2217 () => {
2218         write(csv([[ "编号", "色温", "功率(W)", "数量", ...state.lights.map(l=
2219 >[l.code,l.cct,l.watt,l.qty]]]));
2220     });
2221     },
2222     },
2223     };
2224 [padding]
2225     function setActiveNav(hash) {
2226         document.querySelectorAll(".nav a").forEach((a) => {
2227             const target = (a.getAttribute("href") || "").replace(/^#/ , "");
2228             a.classList.toggle("active", target === hash);
2229         });
2230     }
2231 [padding]
2232     function currentHash() {
2233         const h = (location.hash || "").replace(/^#/ , "");
2234         return h || "home";
2235     }
2236 [padding]
2237     function ensureAuth() {
2238         const view = currentHash();
2239         if (!state.token && view !== "login") {
2240             location.hash = "login";
2241             return false;
2242         }
2243         return true;
2244     }
2245 [padding]
2246     function openModal({ title, bodyHtml, footerButtons }) {
2247         const shell = document.getElementById("modal");
2248         shell.classList.add("modal");
2249         shell.innerHTML = `
2250         <div class="modal" role="dialog" aria-label="${escapeHtml(title)}">

```

```

2251     <div class="mhd">
2252         <strong>${escapeHtml(title)}</strong>
2253         <button class="btn" id="modal-close">关闭</button>
2254     </div>
2255     <div class="mbd">${bodyHtml}</div>
2256     <div class="mft">${footerButtons || `<button class="btn primary" id=
2257 "modal-ok">确定</button>`}</div>
2258     </div>
2259     `;
2260     shell.querySelector("#modal-close").addEventListener("click",
2261 closeModal);
2262     shell.querySelector("#modal-ok").addEventListener("click", closeModal);
2263     }
2264 [padding]
2265     function closeModal() {
2266         const shell = document.getElementById("modal");
2267         shell.classList.remove("modal");
2268         shell.innerHTML = "";
2269     }
2270 [padding]
2271     function openPlantModal(action) {
2272         const map = { create: "新增植物品种", edit: "编辑植物信息", view:
2273 "查看植物详情", delete: "删除植物确认" };
2274         const title = map[action] || "植物操作";
2275         if (action === "delete") {
2276             return openModal({
2277                 title,
2278                 footerButtons: `<button class="btn" id="modal-
2279 ok">取消</button><button class="btn danger" id="modal-
2280 del">确认删除</button>`,
2281             });
2282         }
2283         const readonly = action === "view" ? "readonly" : "";
2284         const hint = action === "view" ? "（只读预览）" : "（示例表单）";
2285         openModal({
2286             title: `${title}${hint}`,
2287             bodyHtml: `
2288                 <div class="split">
2289                     <div class="field"><label>中文名</label><input value="紫薇"
2290 ${readonly} /></div>
2291                     <div class="field"><label>类别</label><select ${readonly ?
2292 "disabled" : ""}><option>乔木</option><option>灌木</option><option>地被</opt
2293 ion></select></div>
2294                     <div class="field"><label>常绿/落叶</label><select ${readonly ?
2295 "disabled" : ""}><option>落叶</option><option>常绿</option></select></div>
2296                     <div class="field"><label>冠幅范围</label><input value="3-5m"
2297 ${readonly} /></div>
2298                 </div>
2299                 <div class="field"><label>季相要点</label><textarea ${readonly}
2300 >夏季花期明显，适合节点色彩强调；秋季叶色偏黄，建议与常绿灌木搭配。</textare

```

```

2301 a></div>
2302 ` ,
2303 });
2304 }
2305 [padding]
2306 function escapeHtml(s) {
2307     return String(s).replaceAll("&", "&amp;").replaceAll("<", "&lt;");
2308     .replaceAll(">", "&gt;").replaceAll("'", "&quot;");
2309 }
2310 [padding]
2311 async function apiLogin(username, password) {
2312     const res = await fetch("/api/auth/login", {
2313         method: "POST",
2314         headers: { "Content-Type": "application/json" },
2315         body: JSON.stringify({ username, password }),
2316     });
2317     const data = await res.json().catch(() => ({}));
2318     if (!res.ok || !data.ok) throw new Error(data.error || "login_failed");
2319     return data;
2320 }
2321 [padding]
2322 function bindLogin() {
2323     const form = document.getElementById("login-form");
2324     if (!form) return;
2325     form.addEventListener("submit", async (e) => {
2326         e.preventDefault();
2327         const u = form.querySelector('input[name="username"]').value.trim();
2328         const p = form.querySelector('input[name="password"]').value.trim();
2329         const msg = document.getElementById("login-msg");
2330         try {
2331             const data = await apiLogin(u, p);
2332             state.token = data.token;
2333             state.role = data.role;
2334             localStorage.setItem("g3d_token", state.token);
2335             localStorage.setItem("g3d_role", state.role);
2336             location.hash = "home";
2337         } catch (err) {
2338             if (msg) msg.textContent = "账号或密码不正确（演示：admin/admin123）";
2339         }
2340     });
2341 }
2342 }
2343 [padding]
2344 function renderLogin() {
2345     return `
2346     <div class="wrap">
2347         <section class="card" style="max-width:860px; margin:26px auto;">
2348             <div class="hd">
2349                 <h2>登录</h2>
2350                 <span class="pill">园林景观三维可视化设计平台</span>

```

```
2351     </div>
2352     <div class="bd">
2353         <div class="split">
2354             <div>
2355                 <form id="login-form">
2356                     <div class="field"><label>用户名</label><input name=
2357 "username" autocomplete="username" value="admin" /></div>
2358                     <div class="field"><label>密码</label><input name=
2359 "password" type="password" autocomplete="current-password" value="admin123"
2360 /></div>
2361                     <div class="row">
2362                         <button class="btn primary" type="submit">登录</button>
2363                         <span id="login-msg" class="muted"></span>
2364                     </div>
2365                 </form>
2366             </div>
2367             <div>
2368                 <div class="card" style="box-shadow:none; background:#fbfcfb;
2369 border-style:dashed;">
2370                     <div class="bd">
2371                         <div class="view-title">
2372                             <div>
2373                                 <h3>演示说明</h3>
2374                             </div>
2375                             <span class="pill">V1.0</span>
2376                         </div>
2377                         <ul class="muted">
2378                             <li>设计师: designer/design123</li>
2379                             <li>评审人: reviewer/review123</li>
2380                             <li>管理员: admin/admin123</li>
2381                         </ul>
2382                         <div class="muted">视觉风格: 低饱和绿 + 暖灰 +
2383 克制阴影, 突出绿植与场景留白。</div>
2384                     </div>
2385                 </div>
2386             </div>
2387         </div>
2388     </div>
2389 </section>
2390 </div>
2391 `;
2392 }
2393 [padding]
2394 function renderShell(innerHtml) {
2395     const h = currentHash();
2396     const navItems = [
2397         ["home", "总览"],
2398         ["site", "场地"],
2399         ["plan", "方案"],
2400         ["planting", "植物"],
```



```

2401     ["materials", "材料"],
2402     ["review", "评审"],
2403     ["export", "导出"],
2404 ];
2405 return `
2406 <header>
2407   <div class="topbar">
2408     <div class="brand">
2409       <div class="logo" aria-hidden="true"></div>
2410       <div>
2411         <h1>园林景观三维可视化设计平台</h1>
2412         <div class="sub">自然雅致 · 版本 V1.0 · 登录角色: ${state.role
2413 || "未登录"}</div>
2414       </div>
2415     </div>
2416     <nav class="nav" aria-label="主导航">
2417       ${navItems.map(([k, t]) => `<a href="#${k}" class="${k === h ?
2418 "active" : ""}">${t}</a>`).join("")}
2419       <a href="#login" id="logout-link">退出</a>
2420     </nav>
2421   </div>
2422 </header>
2423 <div class="wrap">${innerHTML}</div>
2424 <div id="modal" class="modal-shell"></div>
2425 `;
2426 }
2427 [padding]
2428 function bindShell() {
2429   document.getElementById("logout-link")?.addEventListener("click", () =>
2430 {
2431     state.token = "";
2432     state.role = "";
2433     localStorage.removeItem("g3d_token");
2434     localStorage.removeItem("g3d_role");
2435     closeModal();
2436   });
2437 }
2438 [padding]
2439 async function render() {
2440   const mount = document.getElementById("app");
2441   const h = currentHash();
2442   if (!mount) return;
2443 [padding]
2444   if (h === "login") {
2445     mount.innerHTML = renderLogin();
2446     bindLogin();
2447     return;
2448   }
2449 [padding]
2450   if (!ensureAuth()) return;

```

```
2451     if (!state.plants.length || !state.materials.length ||
2452 !state.issues.length) {
2453         try {
2454             await loadCoreData();
2455         } catch (e) {
2456             // During static preview, api may be unavailable; keep UI usable.
2457         }
2458     }
2459     const view = views[h] || views.home;
2460     const inner = view.render();
2461     mount.innerHTML = renderShell(inner);
2462     setActiveNav(h);
2463     bindShell();
2464     view.bind?().();
2465 }
2466 [padding]
2467 window.__rkModal = { closeModal };
2468 window.addEventListener("hashchange", render);
2469 [padding]
2470 return { render };
2471 })();
2472 [padding]
2473 window.addEventListener("DOMContentLoaded", () => App.render());
2474 # apps/web/devtools.html
2475 <!doctype html>
2476 <html lang="zh-CN">
2477 <head>
2478     <meta charset="utf-8" />
2479     <meta name="viewport" content="width=device-width, initial-scale=1" />
2480     <title>开发自检 - 园林景观三维可视化设计平台</title>
2481     <link rel="stylesheet" href="/app.css" />
2482 </head>
2483 <body>
2484     <header>
2485         <div class="topbar">
2486             <div class="brand">
2487                 <div class="logo" aria-hidden="true"></div>
2488                 <div>
2489                     <h1>开发自检</h1>
2490                     <div class="sub">用于本地接口与静态资源快速核对</div>
2491                 </div>
2492             </div>
2493             <nav class="nav" aria-label="辅助导航">
2494                 <a href="/index.html#home">返回系统</a>
2495             </nav>
2496         </div>
2497     </header>
2498     <div class="wrap">
2499         <section class="card">
2500             <div class="hd">
```

```

2501     <h2>接口自检</h2>
2502     <span class="muted">不写入数据，仅拉取查看</span>
2503 </div>
2504 <div class="bd">
2505     <div class="row">
2506         <button class="btn primary" id="btn-health">/api/health</button>
2507         <button class="btn" id="btn-report">/api/report</button>
2508         <button class="btn" id="btn-manifest">/api/web/manifest</button>
2509     </div>
2510     <div style="height:12px"></div>
2511     <pre id="out" class="card" style="box-shadow:none;
2512 background:#fbfcfb; border-style:dashed; padding:12px; white-space:pre-wrap;
2513 margin:0;"></pre>
2514 </div>
2515 </section>
2516 [padding]
2517 <section class="card" style="margin-top:14px;">
2518     <div class="hd"><h2>说明</h2><span class=
2519 "pill">本地运行</span></div>
2520     <div class="bd">
2521         <div class="muted">
2522             <p>该页用于检查： </p>
2523             <ul>
2524                 <li>服务是否可用（health）</li>
2525                 <li>项目汇总报告是否可生成（report）</li>
2526                 <li>静态资源清单是否完整（manifest）</li>
2527             </ul>
2528             <p>若用于截图采集，仍以系统入口 <code>index.html</code>
2529 为准。</p>
2530         </div>
2531     </div>
2532 </section>
2533 </div>
2534 <script src="./devtools.js"></script>
2535 </body>
2536 </html>
2537 [padding]
2538 # apps/web/devtools.js
2539 function pretty(obj) {
2540     if (typeof obj === "string") return obj;
2541     try {
2542         return JSON.stringify(obj, null, 2);
2543     } catch {
2544         return String(obj);
2545     }
2546 }
2547 [padding]
2548 async function fetchText(path) {
2549     const res = await fetch(path);
2550     const ct = res.headers.get("content-type") || "";

```

```
2551     if (!res.ok) throw new Error(`${res.status} ${res.statusText}`);
2552     if (ct.includes("application/json")) return pretty(await res.json());
2553     return await res.text();
2554 }
2555 [padding]
2556 function setOut(text) {
2557     const out = document.getElementById("out");
2558     if (!out) return;
2559     out.textContent = text;
2560 }
2561 [padding]
2562 async function bind() {
2563     const map = {
2564         "btn-health": "/api/health",
2565         "btn-report": "/api/report",
2566         "btn-manifest": "/api/web/manifest",
2567     };
2568     for (const [id, path] of Object.entries(map)) {
2569         document.getElementById(id)?.addEventListener("click", async () => {
2570             setOut(`请求中: ${path}`);
2571             try {
2572                 const text = await fetchText(path);
2573                 setOut(text);
2574             } catch (e) {
2575                 setOut(`请求失败: ${path}\n${String(e)}`);
2576             }
2577         });
2578     }
2579 }
2580 [padding]
2581 window.addEventListener("DOMContentLoaded", bind);
2582 [padding]
2583 # apps/web/index.html
2584 <!doctype html>
2585 <html lang="zh-CN">
2586 <head>
2587     <meta charset="utf-8" />
2588     <meta name="viewport" content="width=device-width, initial-scale=1" />
2589     <title>园林景观三维可视化设计平台</title>
2590     <link rel="stylesheet" href="./app.css" />
2591 </head>
2592 <body>
2593     <div id="app"></div>
2594     <script src="./app.js"></script>
2595 </body>
2596 </html>
2597 [padding]
2598 # apps/web/student.html
2599 <!doctype html>
2600 <html lang="zh-CN">
```

```

2601 <head>
2602 <meta charset="utf-8" />
2603 <meta name="viewport" content="width=device-width, initial-scale=1" />
2604 <title>评审浏览 - 园林景观三维可视化设计平台</title>
2605 <meta http-equiv="refresh" content="0; url=./index.html#review" />
2606 </head>
2607 <body>
2608 <p>正在跳转到评审浏览...</p>
2609 </body>
2610 </html>
2611 # apps/web/teacher.html
2612 <!doctype html>
2613 <html lang="zh-CN">
2614 <head>
2615 <meta charset="utf-8" />
2616 <meta name="viewport" content="width=device-width, initial-scale=1" />
2617 <title>设计师工作台 - 园林景观三维可视化设计平台</title>
2618 <meta http-equiv="refresh" content="0; url=./index.html#plan" />
2619 </head>
2620 <body>
2621 <p>正在跳转到设计师工作台...</p>
2622 </body>
2623 </html>
2624 # scripts/ai_slop_check.py
2625 # -*- coding: utf-8 -*-
2626 from __future__ import annotations
2627 [padding]
2628 import runpy
2629 import sys
2630 from pathlib import Path
2631 [padding]
2632 [padding]
2633 def main() -> None:
2634     workspace_root = Path(__file__).resolve().parents[2]
2635     script = workspace_root / "scripts" / "ai_slop_check.py"
2636     if not script.exists():
2637         raise FileNotFoundError(f"Missing workspace script: {script}")
2638     sys.path.insert(0, str(workspace_root / "scripts"))
2639     runpy.run_path(str(script), run_name="__main__")
2640 [padding]
2641 [padding]
2642 if __name__ == "__main__":
2643     main()
2644 [padding]
2645 # scripts/api_self_check.py
2646 # -*- coding: utf-8 -*-
2647 from __future__ import annotations
2648 [padding]
2649 """
2650 API self-check (local).

```

```
2651 [padding]
2652 This script is intentionally simple and dependency-free. It can be used to:
2653 - verify the built-in server starts;
2654 - verify key endpoints return expected shapes;
2655 - run a short smoke cycle before screenshot capture.
2656 [padding]
2657 Note: The gated pipeline uses `scripts/run_tests.py` and
2658 `scripts/build_materials.py`.
2659 This script is optional but useful during maintenance.
2660 """
2661 [padding]
2662 import json
2663 import socket
2664 import time
2665 import urllib.error
2666 import urllib.request
2667 from contextlib import closing
2668 from dataclasses import dataclass
2669 from pathlib import Path
2670 from subprocess import Popen
2671 import sys
2672 import os
2673 [padding]
2674 [padding]
2675 def _find_free_port(start: int = 8010) -> int:
2676     for port in range(start, start + 50):
2677         with closing(socket.socket(socket.AF_INET, socket.SOCK_STREAM)) as
2678 s:
2679             try:
2680                 s.bind(("127.0.0.1", port))
2681                 return port
2682             except OSError:
2683                 continue
2684     raise RuntimeError("no free port found")
2685 [padding]
2686 [padding]
2687 def _get(url: str, timeout: float = 3.0) -> tuple[int, str, str]:
2688     req = urllib.request.Request(url, method="GET")
2689     with urllib.request.urlopen(req, timeout=timeout) as resp:
2690         data = resp.read()
2691         ct = resp.headers.get("content-type", "")
2692         return resp.status, ct, data.decode("utf-8", errors="replace")
2693 [padding]
2694 [padding]
2695 def _post_json(url: str, payload: dict, timeout: float = 3.0) -> tuple[int,
2696 str, str]:
2697     body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
2698     req = urllib.request.Request(url, data=body, method="POST")
2699     req.add_header("Content-Type", "application/json; charset=utf-8")
2700     with urllib.request.urlopen(req, timeout=timeout) as resp:
```

```
2701         data = resp.read()
2702         ct = resp.headers.get("content-type", "")
2703         return resp.status, ct, data.decode("utf-8", errors="replace")
2704 [padding]
2705 [padding]
2706 def _wait_ready(base_url: str, *, seconds: float = 10.0) -> None:
2707     deadline = time.time() + seconds
2708     last_err = None
2709     while time.time() < deadline:
2710         try:
2711             status, _, _ = _get(base_url + "/api/health", timeout=1.5)
2712             if status == 200:
2713                 return
2714         except Exception as exc:
2715             last_err = exc
2716             time.sleep(0.3)
2717     raise RuntimeError(f"server not ready: {last_err}")
2718 [padding]
2719 [padding]
2720 @dataclass(frozen=True)
2721 class Check:
2722     name: str
2723     ok: bool
2724     detail: str
2725 [padding]
2726 [padding]
2727 def _assert_json_ok(text: str) -> dict:
2728     obj = json.loads(text)
2729     if not isinstance(obj, dict) or obj.get("ok") is not True:
2730         raise AssertionError("response not ok")
2731     return obj
2732 [padding]
2733 [padding]
2734 def run_checks(base_url: str) -> list[Check]:
2735     results: list[Check] = []
2736 [padding]
2737     def do(name: str, fn) -> None:
2738         try:
2739             detail = fn()
2740             results.append(Check(name=name, ok=True, detail=detail))
2741         except Exception as exc:
2742             results.append(Check(name=name, ok=False, detail=str(exc)))
2743 [padding]
2744     do("health", lambda: str(_get(base_url + "/api/health")[0]))
2745     do("plants", lambda: str(len(_assert_json_ok(_get(base_url +
2746 "/api/plants")[2]).get("items", []))))
2747     do("materials", lambda: str(len(_assert_json_ok(_get(base_url +
2748 "/api/materials")[2]).get("items", []))))
2749     do("issues", lambda: str(len(_assert_json_ok(_get(base_url +
2750 "/api/issues")[2]).get("items", []))))
```

```

2751     do("report", lambda: "ok" if "关键指标" in _get(base_url + "/api/report")
2752 [2] else "missing")
2753     do("manifest", lambda: "ok" if "manifest" in _get(base_url +
2754 "/api/web/manifest")[2] else "missing")
2755 [padding]
2756     def login_and_export() -> str:
2757         _, t = _post_json(base_url + "/api/auth/login", {"username":
2758 "admin", "password": "admin123"})
2759         obj = _assert_json_ok(t)
2760         token = obj.get("token", "")
2761         if not token:
2762             raise AssertionError("missing token")
2763         status, _, csv_text = _get(base_url + "/api/export?kind=plants")
2764         if status != 200 or "编号" not in csv_text:
2765             raise AssertionError("export not ok")
2766         return "ok"
2767 [padding]
2768     do("login+export", login_and_export)
2769     return results
2770 [padding]
2771 [padding]
2772 def main() -> int:
2773     project_root = Path(__file__).resolve().parents[1]
2774     port = _find_free_port()
2775     env = os.environ.copy()
2776     env["APP_PORT"] = str(port)
2777     server = Popen([sys.executable, "apps/api/main.py"], cwd=project_root,
2778 env=env)
2779     base_url = f"http://127.0.0.1:{port}"
2780     try:
2781         _wait_ready(base_url)
2782         results = run_checks(base_url)
2783     finally:
2784         server.terminate()
2785     try:
2786         server.wait(timeout=5)
2787     except Exception:
2788         server.kill()
2789 [padding]
2790     failed = [r for r in results if not r.ok]
2791     for r in results:
2792         flag = "PASS" if r.ok else "FAIL"
2793         print(f"[{flag}] {r.name}: {r.detail}")
2794     return 1 if failed else 0
2795 [padding]
2796 [padding]
2797 if __name__ == "__main__":
2798     raise SystemExit(main())
2799 [padding]
2800 # scripts/build_materials.py

```



```
2801 # -*- coding: utf-8 -*-
2802 from __future__ import annotations
2803 [padding]
2804 import runpy
2805 import sys
2806 from pathlib import Path
2807 [padding]
2808 [padding]
2809 def main() -> None:
2810     workspace_root = Path(__file__).resolve().parents[2]
2811     script = workspace_root / "scripts" / "build_materials.py"
2812     if not script.exists():
2813         raise FileNotFoundError(f"Missing workspace script: {script}")
2814     sys.path.insert(0, str(workspace_root / "scripts"))
2815     runpy.run_path(str(script), run_name="__main__")
2816 [padding]
2817 [padding]
2818 if __name__ == "__main__":
2819     main()
2820 [padding]
2821 # scripts/count_lines.py
2822 """Count non-empty source-code lines for this project."""
2823 [padding]
2824 from __future__ import annotations
2825 [padding]
2826 from pathlib import Path
2827 [padding]
2828 [padding]
2829 def _resolve_repo_root() -> Path:
2830     cwd = Path.cwd().resolve()
2831     if (cwd / "apps").exists() or (cwd / "packages").exists():
2832         return cwd
2833     return Path(__file__).resolve().parents[1]
2834 [padding]
2835 [padding]
2836 REPO_ROOT = _resolve_repo_root()
2837 SOURCE_ROOTS = [REPO_ROOT / "apps", REPO_ROOT / "packages", REPO_ROOT /
2838 "scripts"]
2839 TARGET_SUFFIXES = {".py", ".js", ".mjs", ".css", ".html", ".sql", ".json"}
2840 EXCLUDED_DIRS = {
2841     ".git",
2842     ".tools",
2843     "__pycache__",
2844     "node_modules",
2845     "run_reports",
2846     "tmp",
2847     "copyright-application-materials",
2848 }
2849 [padding]
2850 [padding]
```

```
2851 def iter_source_files() -> list[Path]:
2852     files: list[Path] = []
2853     for root in SOURCE_ROOTS:
2854         if not root.exists():
2855             continue
2856         for path in root.rglob("*"):
2857             if path.is_dir():
2858                 continue
2859             if any(part in EXCLUDED_DIRS for part in path.parts):
2860                 continue
2861             if path.suffix.lower() not in TARGET_SUFFIXES:
2862                 continue
2863             files.append(path)
2864     return sorted(files)
2865 [padding]
2866 [padding]
2867 def count_lines() -> int:
2868     total = 0
2869     for path in iter_source_files():
2870         try:
2871             content = path.read_text(encoding="utf-8")
2872         except UnicodeDecodeError:
2873             continue
2874         total += sum(1 for line in content.splitlines() if line.strip())
2875     return total
2876 [padding]
2877 [padding]
2878 if __name__ == "__main__":
2879     print(count_lines())
2880 [padding]
2881 # scripts/data_tools.py
2882 # -*- coding: utf-8 -*-
2883 from __future__ import annotations
2884 [padding]
2885 """
2886 Data tools for local JSON state.
2887 [padding]
2888 This module provides tiny maintenance helpers that are safe to run offline:
2889 - validate required keys exist;
2890 - normalize list ordering for stable diffs;
2891 - rebuild minimal seed state if file is missing.
2892 """
2893 [padding]
2894 import json
2895 from dataclasses import dataclass
2896 from pathlib import Path
2897 from typing import Any
2898 [padding]
2899 [padding]
2900 @dataclass(frozen=True)
```

```
2901 class Result:
2902     ok: bool
2903     message: str
2904 [padding]
2905 [padding]
2906 REQUIRED_TOP_KEYS = ["seeded", "sites", "plans", "versions", "plants",
2907 "materials", "lights", "issues", "pavings"]
2908 [padding]
2909 [padding]
2910 def read_state(path: Path) -> dict[str, Any]:
2911     if not path.exists():
2912         return {}
2913     return json.loads(path.read_text(encoding="utf-8"))
2914 [padding]
2915 [padding]
2916 def write_state(path: Path, state: dict[str, Any]) -> None:
2917     path.parent.mkdir(parents=True, exist_ok=True)
2918     path.write_text(json.dumps(state, ensure_ascii=False, indent=2),
2919 encoding="utf-8")
2920 [padding]
2921 [padding]
2922 def validate_state(state: dict[str, Any]) -> Result:
2923     if not isinstance(state, dict):
2924         return Result(False, "state must be an object")
2925     missing = [k for k in REQUIRED_TOP_KEYS if k not in state]
2926     if missing:
2927         return Result(False, "missing keys: " + ", ".join(missing))
2928     for k in REQUIRED_TOP_KEYS[1:]:
2929         if not isinstance(state.get(k), list):
2930             return Result(False, f'{k} must be a list')
2931     return Result(True, "ok")
2932 [padding]
2933 [padding]
2934 def sort_state(state: dict[str, Any]) -> dict[str, Any]:
2935     out = dict(state)
2936     for key in ("sites", "plans", "versions", "plants", "materials",
2937 "lights", "issues", "pavings"):
2938         items = out.get(key)
2939         if not isinstance(items, list):
2940             continue
2941         out[key] = sorted(items, key=lambda it: str((it or {}).get("id", "")))
2942 )
2943     return out
2944 [padding]
2945 [padding]
2946 def main() -> int:
2947     project_root = Path(__file__).resolve().parents[1]
2948     state_path = project_root / "apps" / "api" / "data" /
2949 "project_state.json"
2950     state = read_state(state_path)
```

```
2951     if not state:
2952         print("state file missing or empty; nothing to normalize")
2953         return 0
2954     res = validate_state(state)
2955     if not res.ok:
2956         print("FAIL:", res.message)
2957         return 1
2958     sorted_state = sort_state(state)
2959     write_state(state_path, sorted_state)
2960     print("PASS: normalized", state_path)
2961     return 0
2962 [padding]
2963 [padding]
2964 if __name__ == "__main__":
2965     raise SystemExit(main())
2966 [padding]
2967 # scripts/util_text.py
2968 # -*- coding: utf-8 -*-
2969 from __future__ import annotations
2970 [padding]
2971 [padding]
2972 def wrap_lines(text: str, width: int = 80) -> list[str]:
2973     out: list[str] = []
2974     for raw in (text or "").splitlines():
2975         line = raw.rstrip("\n")
2976         while len(line) > width:
2977             cut = line.rfind(" ", 0, width + 1)
2978             if cut <= 0:
2979                 cut = width
2980             out.append(line[:cut].rstrip())
2981             line = line[cut:].lstrip()
2982         out.append(line)
2983     return out
2984 [padding]
2985 [padding]
2986 def normalize_ws(text: str) -> str:
2987     return " ".join((text or "").split())
2988 [padding]
2989 [padding]
2990 def clamp(n: int, *, lo: int, hi: int) -> int:
2991     return max(lo, min(hi, int(n)))
2992 [padding]
2993 [padding]
2994 def is_blank(text: str | None) -> bool:
2995     return not (text or "").strip()
2996 [padding]
2997 [padding]
2998 def title_case(text: str) -> str:
2999     return " ".join(w[1:].upper() + w[1:] for w in normalize_ws(text)
3000 .split(" ") if w)
```